Mathematical Foundations of Deep Neural Networks

Lecture Notes, Part II: Deep Networks (Ch. 5–9)

Contents

Chapter 5. Neural Networks and Backpropagation 5

1 From Hand-Designed Features to Learned Functions 5

2 Neurons, Layers and Shapes 5

3 Why Nonlinearity and Depth 6

4 Activation Functions and Outputs 6

5 Computation Graphs and the Scalar Chain Rule 6

6 Vector Calculus Rules for Backpropagation 7

6.1 Backpropagation through an MLP 8

7 Worked Results 8

7.1 Single neurons: MSE versus cross-entropy 8

7.2 Residual MLP 8

Chapter 6. Training and Optimization of Deep Networks 10

1 Full, Stochastic and Mini-Batch Gradients 10

2 Activation Functions Revisited 10

3 Weight Initialization 11

3.1 Why the scale matters 11

3.2 Zero and symmetric initialization 11

3.3 Xavier and He initialization 12

3.4 Variance of a sigmoid output 13

4 Learning-Rate Schedules 13

5 Batch Normalization 13

5.1 Training-time computation . . . 13
5.2 Test time . . . 14

6 Dropout **14**

7 Gradient Descent and Taylor's Theorem **15**

8 Smoothness and the Descent Lemma **15**

9 Stochastic Gradient Descent **16**

10 Momentum **16**
10.1 Heavy-ball momentum . . . 16
10.2 Nesterov momentum . . . 17

11 Adaptive Methods **18**
11.1 Exponential moving averages and bias correction . . . 18
11.2 Special cases and invariances . . . 18
11.3 Weight decay is not ℓ_2 regularization . . . 19

Chapter 7. Convolutional Neural Networks and Architectures **20**

1 Why a New Layer **20**

2 Convolution as a Local Dot Product **20**
2.1 A fixed filter: moving average . . . 20
2.2 Single and multiple channels . . . 20
2.3 Parameter counts . . . 21

3 Stride, Padding and Output Size **21**

4 Learning Filters **22**

5 Shift Equivariance **22**

6 1×1 Convolution, Pooling and Global Average Pooling **23**

7 Algebraic Facts about CNN Layers **23**

8 Vocabulary and Costs **24**

9 AlexNet **24**

10 ZFNet and VGG **25**

11 GoogLeNet and the Inception Module **25**
11.1 Naive module . . . 25
11.2 Bottleneck module . . . 25
11.3 Split–transform–merge . . . 26

12 Network in Network: Exact versus Approximate Invariance **26**

13 Why Depth Can Fail, and Residual Networks **26**

14 Saliency Maps **27**

Chapter 8. Universal Approximation, Barron's Theorem and Generalization **28**

1 Three Sources of Error **28**

2 Preliminaries from Analysis **28**
2.1 Measures . . . 28
2.2 Normed spaces and density . . . 28

3 Cybenko's Universal Approximation Theorem **29**
3.1 The functional-analytic lemmas . . . 29
3.2 Density for discriminatory functions . . . 30
3.3 Continuous sigmoidals are discriminatory . . . 30

4 Barron's Theorem: a Dimension-Free Rate **31**
4.1 Fourier preliminaries . . . 31

5 Proof of Barron's Theorem **31**
5.1 Convex hulls and Maurey's lemma . . . 32
5.2 Barron functions lie in a convex hull of half-space indicators . . . 33
5.3 Completing the proof of Theorem 1 . . . 34

6 Generalization and Rademacher Complexity **35**
6.1 Population and empirical risk . . . 35
6.2 Rademacher complexity . . . 35
6.3 Two-layer networks . . . 36

Chapter 9. Recurrent Neural Networks and LSTM **37**

1 Why Recurrent Networks **37**

2 The Recurrence **37**

3 Backpropagation Through Time **38**

3.1 The idea . . . 38

3.2 A worked two-step example . . . 38

3.3 General formulation . . . 39

4 Vanishing and Exploding Gradients **40**

4.1 RNNs are very deep networks . . . 40

4.2 Exploding gradients and gradient clipping . . . 41

4.3 Vanishing gradients . . . 41

5 Long Short-Term Memory **41**

5.1 The LSTM cell . . . 41

5.2 Gradient flow through the cell . . . 42

6 Applications **43**

6.1 Language modeling . . . 43

6.2 Image captioning . . . 44

Chapter 5. Neural Networks and Backpropagation

Basic concepts of neural networks, multilayer perceptrons, backpropagation

1 From Hand-Designed Features to Learned Functions

A classifier maps an observation $\mathbf{x}$ to an output $y = f(\mathbf{x}; \boldsymbol{\theta})$. Classical pipelines fix features by hand (color histograms, filter banks) and learn only a final classifier; deep learning trains the representation and the classifier end to end from labeled data. Two properties of features recur: a feature is *invariant* to a transformation if it does not change, and *equivariant* if it changes in the same way (Chapter 7).

2 Neurons, Layers and Shapes

Definition 5.1: multilayer perceptron

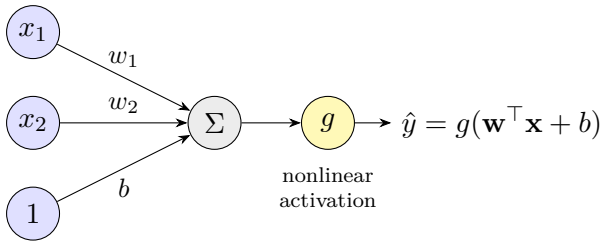
A neuron computes $h = \sigma(\mathbf{w}^\top \mathbf{x} + b)$. A fully connected layer stacks neurons:

$$\mathbf{z}^{(\ell)} = W_\ell \mathbf{a}^{(\ell-1)} + \mathbf{b}_\ell, \qquad \mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)}), \qquad \mathbf{a}^{(0)} = \mathbf{x},$$

with $W_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $\mathbf{b}_\ell \in \mathbb{R}^{n_\ell}$, and σ applied elementwise. An MLP is a composition of such layers; a 1000-neuron layer is one layer, not 1000 layers.

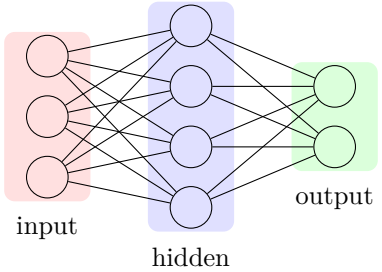
Example 5.2: parameter count

A two-layer classifier $\mathbf{s} = W_2\,\mathrm{ReLU}(W_1\mathbf{x})$ on CIFAR-10 images ($32 \times 32 \times 3 = 3072$ inputs), hidden width 100, 10 classes: $W_1 \in \mathbb{R}^{100\times 3072}$, $W_2 \in \mathbb{R}^{10\times 100}$, so $307{,}200 + 1000 = 308{,}200$ weights (plus 110 biases).



A single neuron: an affine combination of the inputs followed by a nonlinear activation g.

$$f(\mathbf{x}) = \mathrm{softmax}\left(W_2 \max(0, W_1 \mathbf{x})\right)$$



A two-layer network: every unit of a layer is connected to every unit of the next.

3 Why Nonlinearity and Depth

Without activations layers collapse: $W_2(W_1\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2 = (W_2W_1)\mathbf{x}+(W_2\mathbf{b}_1+\mathbf{b}_2)$ is still affine, however many layers there are, and an affine score yields linear decision boundaries. Hidden layers are learned embeddings: e.g. $\phi(x_1,x_2)=(x_1,x_2,x_1^2+x_2^2)$ separates an inner from an outer circle by a plane. A network learns such combinations instead of receiving them by hand; depth composes them hierarchically (strokes $\to$ loops $\to$ digits). Depth increases expressiveness but does not by itself guarantee trainability or generalization (Chapters 6–8).

4 Activation Functions and Outputs

Activation	Definition	Derivative
Sigmoid	$\sigma(z)=(1+e^{-z})^{-1}\in(0,1)$	$\sigma(1-\sigma)\in(0,1/4]$
Tanh	$\tanh z = 2\sigma(2z)-1\in(-1,1)$	$1-\tanh^2 z\in(0,1]$
ReLU	$\max(0,z)$	$\mathbf{1}\{z>0\}$
Leaky ReLU	$\max(az,z)$, $0<a<1$	1 or a
ELU	z $(z\geq 0)$, $a(e^z-1)$ $(z<0)$	1 or ae^z
Maxout	$\max_k(\mathbf{w}_k^\top\mathbf{x}+b_k)$	routes to the max

Outputs: softmax for mutually exclusive classes (loss $-\log p_y$, score gradient $\mathbf{p}-\mathbf{e}_y$), sigmoid for binary probabilities, identity for regression.

Example 5.3: forward pass

$\mathbf{x}=(1,-2)$, $W_1=\begin{pmatrix}1 & 1\\ 2 & -1\end{pmatrix}$, $W_2=\mathbf{I}$, no biases, ReLU hidden layer: $\mathbf{z}_1=(-1,4)$, $\mathbf{a}_1=(0,4)$, scores $(0,4)$, softmax $\approx(0.018,0.982)$.

5 Computation Graphs and the Scalar Chain Rule

Training repeats: zero the gradients, forward pass, loss, backward pass (backpropagation computes derivatives), optimizer update. Backpropagation applies the chain rule on a computation graph: each node multiplies the *upstream* derivative (from later operations) by its *local* derivative.

Example 5.4: $f=(x+y)z$

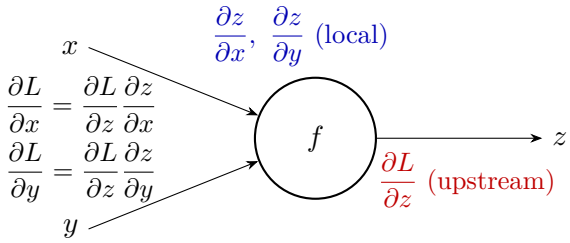
At $(x,y,z)=(-2,5,-4)$: $q=x+y=3$, $f=qz=-12$. Local derivatives $\partial f/\partial q=z=-4$, $\partial f/\partial z=q=3$, $\partial q/\partial x=\partial q/\partial y=1$, hence $\partial f/\partial x=\partial f/\partial y=-4$, $\partial f/\partial z=3$.

Example 5.5: sigmoid neuron

$f=\sigma(w_0x_0+w_1x_1+w_2)$ with $w_0=2, x_0=-1, w_1=-3, x_1=-2, w_2=-3$: $z=1$, $f\approx 0.731$, $\partial f/\partial z=f(1-f)\approx 0.197$. Then $\partial f/\partial w_0=0.197x_0=-0.197$, $\partial f/\partial x_0=0.197w_0=0.393$, $\partial f/\partial w_1=-0.393$, $\partial f/\partial x_1=-0.590$, $\partial f/\partial w_2=0.197$. Decomposing σ into $z\to -z\to e^{-z}\to 1+e^{-z}\to 1/(\cdot)$ gives the same numbers; treating σ as one node is cheaper.

Remark 5.6: gate patterns

Add gate: distributes the upstream derivative to both inputs. Multiply gate: each input receives upstream $\times$ the other input. Copy (a value used twice): derivatives add. Max gate: routes the derivative to the maximal input. Hence a shared parameter accumulates gradients from all its uses.



Backpropagation at one node: the downstream gradient is the upstream gradient times the local gradient.

6 Vector Calculus Rules for Backpropagation

Motivation. Backpropagation is the chain rule organized so that each layer passes one vector backwards. A handful of vector rules (for linear maps, elementwise nonlinearities and losses) suffice to differentiate any feedforward network.

Gradient convention: $\nabla_{\mathbf{x}} L$ has the shape of $\mathbf{x}$; the Jacobian of $\mathbf{y} \in \mathbb{R}^m$ in $\mathbf{x} \in \mathbb{R}^n$ is $m \times n$ with entries $\partial y_i / \partial x_j$.

Theorem 5.7: rules R1–R6

R1. Sum rule: the Jacobian of a sum is the sum of Jacobians; terms not depending on the variable contribute 0.

R2. Identity: $\partial \mathbf{x} / \partial \mathbf{x} = \mathbf{I}$.

R3. Chain rule: $\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ (outer Jacobian on the left).

R4. Affine map: $\partial(A\mathbf{x} + \mathbf{b}) / \partial \mathbf{x} = A$.

R5. Elementwise map: $\partial \sigma(\mathbf{z}) / \partial \mathbf{z} = \operatorname{diag}(\sigma'(\mathbf{z}))$, so $\operatorname{diag}(\sigma'(\mathbf{z}))\mathbf{v} = \sigma'(\mathbf{z}) \odot \mathbf{v}$.

R6. Gradient chain rule (scalar L): $\nabla_{\mathbf{x}} L = \left(\frac{\partial \mathbf{u}}{\partial \mathbf{x}}\right)^{\top} \nabla_{\mathbf{u}} L$.

Proof. R6: by R3 the row vector $\partial L / \partial \mathbf{x} = (\partial L / \partial \mathbf{u})(\partial \mathbf{u} / \partial \mathbf{x})$; transposing reverses the order. ■

Theorem 5.8: affine layer

For $\mathbf{z} = W\mathbf{a} + \mathbf{b}$ and $\boldsymbol{\delta} = \nabla_{\mathbf{z}} L$:

$$\nabla_W L = \boldsymbol{\delta}\, \mathbf{a}^{\top}, \qquad \nabla_{\mathbf{b}} L = \boldsymbol{\delta}, \qquad \nabla_{\mathbf{a}} L = W^{\top} \boldsymbol{\delta}.$$

For an activation layer $\mathbf{h} = \sigma(\mathbf{z})$: $\nabla_{\mathbf{z}} L = \nabla_{\mathbf{h}} L \odot \sigma'(\mathbf{z})$.

Proof. $\nabla_{\mathbf{a}} L$: R6 with R4. $\nabla_{\mathbf{b}} L$: R6 with $\partial \mathbf{z} / \partial \mathbf{b} = \mathbf{I}$ (R1, R2). $\nabla_W L$: row i of W, $\mathbf{w}_i^{\top}$, affects L only through $z_i = \mathbf{w}_i^{\top} \mathbf{a} + b_i$, whose gradient in $\mathbf{w}_i$ is $\mathbf{a}$ (R4); thus $\nabla_{\mathbf{w}_i} L = \delta_i \mathbf{a}$, and stacking rows gives $\boldsymbol{\delta} \mathbf{a}^{\top}$. Equivalently, componentwise $\partial L / \partial W_{ij} = \delta_i a_j$. Activation: R6 with R5, and a diagonal matrix is symmetric. ■

Remark 5.9: differentials and trace

Matrix gradients can also be read from differentials: if $dL = \operatorname{tr}(K\,dW)$ then $\nabla_W L = K^\top$. For $\mathbf{z} = W\mathbf{a}$, $dL = \boldsymbol{\delta}^\top (dW)\mathbf{a} = \operatorname{tr}(\mathbf{a}\boldsymbol{\delta}^\top dW)$, so $\nabla_W L = \boldsymbol{\delta}\mathbf{a}^\top$.

6.1 Backpropagation through an MLP

With $\boldsymbol{\delta}_\ell = \nabla_{\mathbf{z}^{(\ell)}} L$ (an $n_\ell \times 1$ column), the recursion is

$$\boldsymbol{\delta}_\ell = (W_{\ell+1}^\top \boldsymbol{\delta}_{\ell+1}) \odot \sigma'(\mathbf{z}^{(\ell)}), \qquad \nabla_{W_\ell} L = \boldsymbol{\delta}_\ell\, \mathbf{a}^{(\ell-1)\top}, \qquad \nabla_{\mathbf{b}_\ell} L = \boldsymbol{\delta}_\ell.$$

It follows from applying R6 twice: through the affine map ($W_{\ell+1}^\top$) and through the activation (elementwise product). In a batch, outer products are summed (or averaged) over samples.

Example 5.10: forward example continued backward

Take class 1 as correct: $\boldsymbol{\delta}_2 = \mathbf{p} - \mathbf{e}_1 \approx (-0.982, 0.982)$, $\nabla_{W_2} L = \boldsymbol{\delta}_2 \mathbf{a}_1^\top = \begin{pmatrix} 0 & -3.928 \\ 0 & 3.928 \end{pmatrix}$. The first hidden pre-activation was negative, so $\boldsymbol{\delta}_1 = (0, 0.982)$ and $\nabla_{W_1} L = \boldsymbol{\delta}_1 \mathbf{x}^\top = \begin{pmatrix} 0 & 0 \\ 0.982 & -1.964 \end{pmatrix}$: the inactive unit receives no gradient (a "dead ReLU" for this input). For elementwise ReLU, upstream $(4, -1, 5, 9)$ at $\mathbf{x} = (1, -2, 3, -1)$ becomes $(4, 0, 5, 0)$ with no Jacobian ever formed.

7 Worked Results

Motivation. Two standard computations show what the rules reveal: why cross-entropy trains a sigmoid neuron faster than squared error, and how a residual connection keeps the backward signal from vanishing.

7.1 Single neurons: MSE versus cross-entropy

Theorem 5.11: gradient bounds

For $y = \sigma(wx + b)$ and $C = \frac{1}{2}(y - z)^2$: $\nabla C = y(1-y)(y-z)(x, 1)$ and $\|\nabla C\| < \frac{1}{4}\sqrt{1 + x^2}\,(1 + |z|)$. For cross-entropy $C = -[z \log y + (1 - z) \log(1 - y)]$: $\nabla C = (y - z)(x, 1)$.

Proof. Chain rule with $\sigma' = y(1 - y) \le 1/4$ and $|y - z| \le |y| + |z| < 1 + |z|$ since $0 < y < 1$. For cross-entropy, $\partial C/\partial t = -[z(1 - y) - (1 - z)y] = y - z$: the factor σ' cancels. ■

For a saturated wrong neuron ($z = 1$, $wx + b = -10$), the MSE gradient carries the factor $\sigma' \approx 4.5 \times 10^{-5}$ while the cross-entropy gradient is of order one: MSE with sigmoid suffers a learning slowdown. GD reads $(w_{n+1}, b_{n+1}) = (w_n, b_n) - \eta \nabla C$.

7.2 Residual MLP

Consider $y_\ell = \sigma(A_\ell y_{\ell-1} + b_\ell) + y_{\ell-1}$ for $2 \le \ell \le L - 1$, $y_1 = \sigma(A_1 x + b_1)$, $y_L = A_L y_{L-1} + b_L$ (scalar). Put $z_\ell = A_\ell y_{\ell-1} + b_\ell$, $D_\ell = \operatorname{diag}(\sigma'(z_\ell))$ and $g_\ell = \partial y_L / \partial y_\ell \in \mathbb{R}^{1 \times m}$.

Theorem 5.12: residual Jacobians and gradients

$$\frac{\partial y_\ell}{\partial y_{\ell-1}} = I + D_\ell A_\ell \ (2 \le \ell \le L-1), \qquad \frac{\partial y_L}{\partial y_{L-1}} = A_L, \qquad g_\ell = A_L \prod_{j=L-1}^{\ell+1} (I + D_j A_j),$$

$$\frac{\partial y_L}{\partial b_\ell} = g_\ell D_\ell, \qquad \frac{\partial y_L}{\partial A_\ell} = D_\ell g_\ell^\top y_{\ell-1}^\top \quad \left(\tfrac{\partial y_L}{\partial (A_\ell)_{ij}} = (g_\ell)_i \sigma'((z_\ell)_i)(y_{\ell-1})_j\right),$$

and $\partial y_L / \partial b_L = 1$, $\partial y_L / \partial A_L = y_{L-1}^\top$.

Proof. R1 splits y_ℓ into $\sigma(z_\ell)$ and $y_{\ell-1}$; R2 gives I; R3 with R5, R4 gives $D_\ell A_\ell$. For b_ℓ, the residual term does not depend on b_ℓ and $\partial z_\ell / \partial b_\ell = I$, so $\partial y_\ell / \partial b_\ell = D_\ell$; chain with g_ℓ. For A_ℓ, the ith row affects y_L only through $(z_\ell)_i$, giving the componentwise formula; it factorizes into an i-part and a j-part, i.e. an outer product. ■

If $A_j = 0$ or $\sigma'(z_j) = 0$ for some intermediate j, the factor is $I + 0 = I$, not 0: the identity path carries the gradient, so lower-layer gradients need not vanish. Without the residual, the factor is $D_j A_j = 0$ and all lower gradients are exactly zero. Expanding $\prod(I + D_j A_j) = I + \sum D_j A_j + \cdots$ exhibits the skip path explicitly (Chapter 7).

Chapter 6. Training and Optimization of Deep Networks

Mini-batch SGD, activations, weight initialization, learning rates, batch normalization; Taylor expansion and the descent lemma, SGD, momentum, Nesterov, AdaGrad, RMSProp, Adam

1 Full, Stochastic and Mini-Batch Gradients

Motivation. Training minimizes an average loss over millions of examples, and computing the exact gradient at each step is too expensive. Gradients on small random batches are unbiased and cheap, which is the basis of all practical training.

For $f(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} f_i(\boldsymbol{\theta})$, the full gradient costs N per-sample gradients. Choosing I uniformly, $\mathbf{g} = \nabla f_I$ is **unbiased**: $\mathbb{E}[\mathbf{g} \mid \boldsymbol{\theta}] = \frac{1}{N}\sum_i \nabla f_i = \nabla f$. A mini-batch averages b such gradients.

Theorem 6.1: mini-batch variance

If the b indices are drawn independently with replacement and a single draw has conditional covariance Σ, the mini-batch gradient has covariance Σ/b. Without replacement it is $\frac{N-b}{b(N-1)}\Sigma$ (with Σ normalized by N).

Proof. With replacement the b terms are iid with covariance Σ; the covariance of their average is $\frac{1}{b^2}\cdot b\Sigma$. Without replacement the pairwise covariances are $-\Sigma/(N-1)$, which gives the finite-population correction. ■

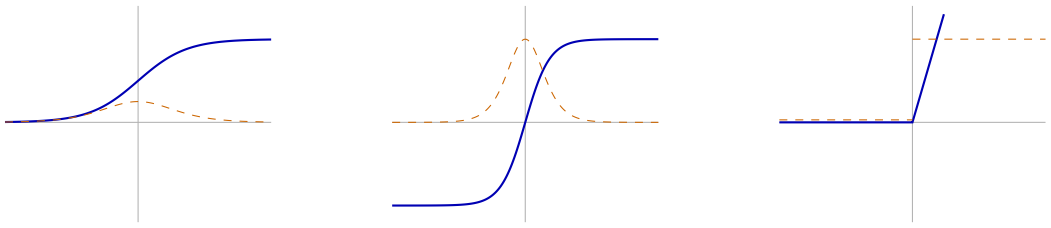
Example 6.2: unbiased but noisy

$f_i(w) = \frac{1}{2}(w - y_i)^2$ with $\mathbf{y} = (1, 3, 5)$ at $w = 0$: per-sample gradients $-1, -3, -5$, full gradient -3. A single random gradient averages to -3; a two-sample batch without replacement gives $-2, -3, -4$, again averaging to -3.

One *iteration* is one parameter update; one *epoch* is one pass through the data (1000 samples, batch 100: 10 iterations per epoch). Full batch: deterministic but expensive; single sample: cheap but noisy; mini-batch: a compromise.

2 Activation Functions Revisited

Sigmoid. Saturates: $\sigma' \le 1/4$ and $\sigma' \to 0$ for large $|z|$. Outputs are positive, so for the next layer $h = W\mathbf{z}$ (with $\mathbf{z}$ the positive sigmoid outputs) $\partial L/\partial W_{ij} = \delta_i z_j$ has the sign of δ_i for every j: all weights into a neuron move in the same direction in one step, producing zig-zag paths. **Tanh.** Zero-centered and odd, $\tanh' \in (0, 1]$, but still saturates near ± 1. **ReLU.** Cheap, derivative 1 on the positive side (no saturation there), but units with negative input get zero gradient (dead ReLU) and outputs are not centered. Leaky ReLU, ELU and maxout keep a nonzero slope or add flexibility.



sigmoid tanh ReLU

Activation functions (solid) and their derivatives (dashed): the sigmoid and tanh *saturate, ReLU has derivative* 0 *or* 1.

3 Weight Initialization

3.1 Why the scale matters

`W = s * np.random.randn(Din, Dout)` draws iid $N(0, s^2)$ weights. For $z_j = \sum_{i=1}^{D_{\text{in}}} W_{ji} x_i$ with zero-mean independent weights of variance σ_W^2, independent of inputs with $\mathbb{E}x_i^2 = q$,

$$\mathbb{E}z_j = 0, \qquad \operatorname{Var} z_j = D_{\text{in}} \sigma_W^2 q$$

(cross terms vanish because $\mathbb{E}[W_{ji}W_{jk}] = 0$ for $i \neq k$). For six tanh layers of width 4096:

- $s = 0.01$: $D_{\text{in}} s^2 = 0.41$; activations shrink towards 0 layer by layer, and weight gradients $\propto$ inputs become tiny.
- $s = 0.05$: $D_{\text{in}} s^2 = 10.24$; pre-activations have standard deviation ≈ 3, tanh saturates (histograms pile up at ± 1), $\tanh' \approx 0$, and the backpropagated signal $\partial L/\partial \mathbf{z} = \sigma'(\cdot) W^\top \partial L/\partial \mathbf{h}$ is almost zero: no learning, although the loss is still large. A zero gradient caused by saturation is not a sign of optimality.

3.2 Zero and symmetric initialization

Theorem 6.3: zero initialization freezes a tanh MLP

For $f(\mathbf{x}) = A_L y_{L-1} + b_L$, $y_\ell = \tanh(A_\ell y_{\ell-1} + b_\ell)$, all parameters initialized to 0 and GD on $\frac{1}{2}\sum_i (f(X_i) - Y_i)^2$: $A_1, \ldots, A_L, b_1, \ldots, b_{L-1}$ stay 0 forever, and $b_L \to \bar{Y}$ if $0 < \eta < 2/N$.

Proof. Induction on the claim "all parameters except b_L are zero". Forward: $\tanh(0) = 0$ so every hidden output is 0 and $f = b_L$. Backward: $\partial/\partial A_L = \sum_i \delta_L^{(i)} y_{L-1}^\top = 0$; $\boldsymbol{\delta}_{L-1} = (A_L^\top \delta_L) \odot \tanh'(0) = 0$ since $A_L = 0$, and inductively all $\boldsymbol{\delta}_\ell = 0$, so $\partial A_\ell = \boldsymbol{\delta}_\ell y_{\ell-1}^\top = 0$, $\partial b_\ell = 0$. Hence GD does not move them. For b_L: $J = \frac{1}{2}\sum (b_L - Y_i)^2$, $b^{(t+1)} - \bar{Y} = (1 - \eta N)(b^{(t)} - \bar{Y})$. ■

Theorem 6.4: duplicate neurons stay duplicates

For $f(x) = \sum_{j=1}^{p} u_j \sigma(a_j x + b_j)$ trained by Adam (or GD, momentum, RMSProp), if $(a_{p-1}, b_{p-1}, u_{p-1}) = (a_p, b_p, u_p)$ at initialization, they remain equal for all iterations.

Proof. $\partial f/\partial u_j = \sigma(a_j x + b_j)$, $\partial f/\partial a_j = u_j \sigma'(a_j x + b_j) x$, $\partial f/\partial b_j = u_j \sigma'(a_j x + b_j)$; the loss gradient multiplies these by $\partial \ell/\partial f$, which is shared by all neurons. So equal parameters give equal gradient coordinates. Induct on the statement "the paired coordinates of $\boldsymbol{\theta}^k$, m_1^k, m_2^k agree": it holds at $k = 0$ ($m_1^0 = m_2^0 = 0$), and every Adam operation (EMAs, bias corrections, elementwise division) acts coordinatewise. The optimizer state must be part of the induction hypothesis. ■

Hence identical neurons compute the same function forever: random initialization is needed to break symmetry.

3.3 Xavier and He initialization

For an approximately linear activation, preserving the second moment requires $D_{\text{in}}\sigma_W^2 = 1$, i.e. $\sigma_W = 1/\sqrt{D_{\text{in}}}$ (often called "Xavier"; Glorot's compromise $2/(D_{\text{in}} + D_{\text{out}})$ also balances the backward pass).

Theorem 6.5: He initialization

Let $y^+ = A\mathbf{x}$, $\mathbf{x} = \text{ReLU}(\mathbf{y})$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$, where $\mathbf{y}$ is independent of A, its coordinates have mean 0, variance 1, are uncorrelated, and $\mathbf{y} \overset{d}{=} -\mathbf{y}$. Then

(i) $\mathbb{E}x_j^2 = \frac{1}{2}$; (ii) $\mathbb{E}y^+ = 0$; (iii) $\mathbb{E}(y_i^+)^2 = \frac{n_{\text{in}}\sigma_A^2}{2}$;

(iv) $y_1^+, \dots, y_{n_{\text{out}}}^+$ are uncorrelated; (v) $y^+ \overset{d}{=} -y^+$.

Thus $\sigma_A^2 = 2/n_{\text{in}}$ preserves the second moment through ReLU layers.

Proof. (i) $\text{ReLU}(y)^2 = y^2\mathbf{1}\{y > 0\}$; symmetry makes the positive and negative halves of $\mathbb{E}y^2 = 1$ equal ($y = 0$ contributes nothing).

(ii) Independence and $\mathbb{E}A_{ij} = 0$.

(iii) $\mathbb{E}(y_i^+)^2 = \sum_{j,k} \mathbb{E}[A_{ij}A_{ik}]\mathbb{E}[x_j x_k] = \sum_j \sigma_A^2 \mathbb{E}x_j^2$.

(iv) For $i \neq k$ all $\mathbb{E}[A_{ij}A_{kl}] = 0$.

(v) $(A, \mathbf{y}) \overset{d}{=} (-A, \mathbf{y})$ since A is independent of $\mathbf{y}$ and symmetric; apply $F(A, \mathbf{y}) = A\,\text{ReLU}(\mathbf{y})$. ■

Remark 6.6: refinements

Leaky ReLU with slope α: $\mathbb{E}\phi(y)^2 = \frac{1+\alpha^2}{2}$ and $\sigma_A^2 = \frac{2}{(1+\alpha^2)n_{\text{in}}}$. ReLU output is not centered: for $z \sim N(0, v)$, $\mathbb{E}\,\text{ReLU}(z) = \sqrt{v/(2\pi)}$, $\mathbb{E}\,\text{ReLU}(z)^2 = v/2$, so He preserves the second moment, not the variance. Backward, $\partial\ell/\partial\mathbf{x} = A^\top \partial\ell/\partial\mathbf{y}^+$ has variance $n_{\text{out}}\sigma_A^2$: forward and backward preservation ask for n_{in} versus n_{out}.

Theorem 6.7: sigmoid layer

For $\mathbf{y} = \sigma(A\mathbf{x})$ with $\sigma(t) \approx \frac{1}{2} + \frac{t}{4}$, $\sigma' \approx \frac{1}{4}$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$ independent of $\mathbf{x}$, and x_j uncorrelated with mean $\frac{1}{2}$ and variance 1: $\mathbb{E}y_i = \frac{1}{2}$, $\text{Var}\, y_i = \frac{5n_{\text{in}}\sigma_A^2}{64}$. If the upstream gradients $\partial\ell/\partial y_i$ have mean 0, variance 1, are uncorrelated and independent of A, then $\partial\ell/\partial x_j \approx \frac{1}{4}\sum_i A_{ij}\partial\ell/\partial y_i$ has mean 0 and variance $\frac{n_{\text{out}}\sigma_A^2}{16}$.

Proof. $\text{Var}(A_{ij}x_j) = \mathbb{E}A_{ij}^2\mathbb{E}x_j^2 = \sigma_A^2(1 + \frac{1}{4})$ and the terms are uncorrelated; multiply by $\frac{1}{16}$. The backward computation is identical with $\mathbb{E}(\partial\ell/\partial y_i)^2 = 1$. ■

Theorem 6.8: convolution layer

For a 3×3, padding-1 convolution $Y_{cij} = \sum_{p,q,r} w_{cpqr}\tilde{X}_{p,i+q-2,j+r-2} + b_c$ with zero-mean unit-variance uncorrelated inputs, $w \overset{\text{iid}}{\sim} N(0, \sigma_w^2)$, $b \overset{\text{iid}}{\sim} N(0, \sigma_b^2)$ independent: $\mathbb{E}Y_{cij} = 0$

everywhere, and $\mathbb{E}Y_{cij}^2 = \sigma_w^2 \cdot \#\{\text{in-range inputs}\} + \sigma_b^2$, i.e. $9C_{\text{in}}\sigma_w^2 + \sigma_b^2$ in the interior, $6C_{\text{in}}\sigma_w^2 + \sigma_b^2$ on edges and $4C_{\text{in}}\sigma_w^2 + \sigma_b^2$ at corners.

Proof. Expand the square: weight–weight terms leave $\sigma_w^2 \mathbb{E}\tilde{X}_a^2$ on the diagonal, weight–bias terms vanish, and $\mathbb{E}b_c^2 = \sigma_b^2$. Padded entries are zero. ■

3.4 Variance of a sigmoid output

For smooth f and X with mean μ, Taylor's theorem $f(X) = f(\mu) + f'(\mu)(X-\mu) + \frac{1}{2}f''(\xi)(X-\mu)^2$ gives $\operatorname{Var} f(X) = f'(\mu)^2 \operatorname{Var} X + \text{Error}$, the error collecting terms with f''. For $X \sim N(0,1)$ and $Y = \sigma(wX + b)$, $\operatorname{Var} Y \approx w^2\sigma'(b)^2$: small weights or large $|b|$ (saturation) give small output variance. An exact bound also holds: if f is L-Lipschitz, $\operatorname{Var} f(X) \le \mathbb{E}(f(X) - f(\mu))^2 \le L^2 \operatorname{Var} X$, so $\operatorname{Var} Y \le w^2/16$.

4 Learning-Rate Schedules

The learning rate controls the update size: too large diverges, too small is slow, and a large early rate followed by a smaller late rate refines the solution and reduces gradient noise.

- Step: multiply by 0.1 at fixed epochs (ResNet: 30, 60, 90), giving $\eta_0, 0.1\eta_0, 0.01\eta_0, 0.001\eta_0$.
- Cosine: $\eta_t = \frac{\eta_0}{2}[1 + \cos(\pi t/T)]$; linear: $\eta_t = \eta_0(1 - t/T)$; inverse square root: $\eta_t = \eta_0/\sqrt{t}$.
- Linear warmup: increase from 0 to $\eta_{\max}$ over the first iterations (e.g. 5000), then decay.

5 Batch Normalization

5.1 Training-time computation

For a mini-batch $X \in \mathbb{R}^{B \times D}$ (rows: samples, columns: features),

$$\mu_j = \frac{1}{B}\sum_i X_{ij}, \quad v_j = \frac{1}{B}\sum_i (X_{ij} - \mu_j)^2, \quad \hat{X}_{ij} = \frac{X_{ij} - \mu_j}{\sqrt{v_j + \epsilon}}, \quad Y_{ij} = \gamma_j \hat{X}_{ij} + \beta_j.$$

γ, β are learned, so the network can choose the output scale and center (with $\gamma_j = \sqrt{v_j + \epsilon}$, $\beta_j = \mu_j$ it recovers the input for that batch).

Example 6.9: two features

$X = \begin{pmatrix} 1 & 10 \\ 3 & 14 \end{pmatrix}$: $\mu = (2, 12)$, $v = (1, 4)$, $\hat{X} = \begin{pmatrix} -1 & -1 \\ 1 & 1 \end{pmatrix}$ (ϵ ignored). With $\gamma = (2, 3)$, $\beta = (-1, 5)$, $Y = \begin{pmatrix} -3 & 2 \\ 1 & 8 \end{pmatrix}$.

Theorem 6.10: invariances

With $\epsilon = 0$:
(i) each column of $\hat{X}$ has mean 0 and mean square 1;

(ii) for $z = \mathbf{w}^\top \mathbf{x} + b$ the BN output does not depend on b and is invariant under $\mathbf{w} \mapsto c\mathbf{w}$, $c > 0$;
(iii) consequently $\langle \nabla_{\mathbf{w}} L, \mathbf{w} \rangle = 0$.

Proof. (ii) $z - \mu = \mathbf{w}^\top(\mathbf{x} - \bar{\mathbf{x}})$ removes b; scaling $\mathbf{w}$ scales both $z - \mu$ and $\sqrt{v}$ by c.
(iii) $h(c) = L(c\mathbf{w})$ is constant, so $0 = h'(1) = \nabla L(\mathbf{w})^\top \mathbf{w}$. ■

5.2 Test time

At inference the batch may have size one, and the output must not depend on the other samples. During training BN keeps running averages

$$\mu^{\text{run}} \leftarrow m\,\mu^{\text{run}} + (1-m)\mu_{\text{batch}}, \qquad (\sigma^2)^{\text{run}} \leftarrow m\,(\sigma^2)^{\text{run}} + (1-m)\sigma^2_{\text{batch}}$$

(an exponential moving average, as in Adam; PyTorch's `momentum` argument is $1 - m$), and at test time uses

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j^{\text{run}}}{\sqrt{(\sigma_j^2)^{\text{run}} + \epsilon}}, \qquad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j.$$

With a batch of one and batch statistics, $\mu = x$ and $v = 0$ would give $\hat{x} = 0$: all information lost. In code: `model.train()` versus `model.eval()`.

Theorem 6.11: fusion at inference

At test time BN is the affine map $z \mapsto az + c$ with $a_j = \gamma_j / \sqrt{v_j^{\text{run}} + \epsilon}$, $c_j = \beta_j - a_j \mu_j^{\text{run}}$, so $\text{BN}(W\mathbf{x} + \mathbf{b}) = \text{diag}(\mathbf{a})W\mathbf{x} + (\text{diag}(\mathbf{a})\mathbf{b} + \mathbf{c})$ can be folded into the preceding layer.

Remark 6.12: why BN helps

Reported benefits: easier optimization, better gradient flow, larger learning rates, less sensitivity to initialization, mild regularization through batch noise. The original "internal covariate shift" explanation has been challenged; a smoother loss landscape is an alternative view. In training mode BN couples samples in the backward pass: with $g_i = \partial L / \partial y_i$ and $r = \sqrt{v + \epsilon}$, $\partial L / \partial x_i = \frac{\gamma}{r}(g_i - \bar{g} - \hat{x}_i\, \overline{g\hat{x}})$.

6 Dropout

Inverted dropout multiplies each unit by $m_i/(1-p)$ with $m_i \sim \text{Bernoulli}(1-p)$ during training (so $\mathbb{E}$ = identity) and does nothing at evaluation. It prevents co-adaptation and acts like training an ensemble of sub-networks (AlexNet applies it to the FC layers).

Theorem 6.13: dropout and activations commute iff positively homogeneous

With the same mask, $\phi(H \odot \mathbf{z}) = H \odot \phi(\mathbf{z})$ for all masks iff $\phi(0) = 0$ and $\phi(cz) = c\phi(z)$ for $c = 1/(1-p)$. This holds for ReLU and LeakyReLU, but fails for the sigmoid.

Proof. Coordinatewise, a dropped unit gives $\phi(0)$ versus 0, a kept unit $\phi(cz)$ versus $c\phi(z)$. For $c > 0$, ReLU and LeakyReLU preserve sign cases and scale linearly. For the sigmoid with $p = 0.4$ and $z = 0$: $\sigma(0)/0.6 \approx 0.833 \neq \sigma(0) = 0.5$. ■

7 Gradient Descent and Taylor's Theorem

Motivation. Why does stepping against the gradient decrease the loss? A first-order Taylor expansion says that for a small enough step it must; the rest of the chapter quantifies "small enough".

We minimize $f : \mathbb{R}^d \to \mathbb{R}$; the iterate $\mathbf{x}_t$ is the whole parameter vector. GD is $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$, and $-\nabla f$ is the steepest-descent direction since $\nabla f^\top \mathbf{v} \geq -\|\nabla f\|$ for unit $\mathbf{v}$ (Cauchy–Schwarz).

Theorem 6.14: Taylor along a segment

With $\mathbf{d} = \mathbf{y} - \mathbf{x}$ and $h(t) = f(\mathbf{x} + t\mathbf{d})$,

$$f(\mathbf{y}) - f(\mathbf{x}) = \int_0^1 \nabla f(\mathbf{x}+t\mathbf{d})^\top \mathbf{d}\, dt, \qquad f(\mathbf{y}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{d} + \int_0^1 (1-t)\, \mathbf{d}^\top \nabla^2 f(\mathbf{x}+t\mathbf{d})\mathbf{d}\, dt.$$

Proof. The first is $h(1) - h(0) = \int_0^1 h'$. For the second, $h(1) = h(0) + h'(0) + \int_0^1 \int_0^t h''(s)\, ds\, dt$ and exchanging the order over $0 \leq s \leq t \leq 1$ gives $\int_0^1 (1-s) h''(s)\, ds$. ∎

8 Smoothness and the Descent Lemma

Motivation. If the gradient is L-Lipschitz, the loss lies below a quadratic around each point. This descent lemma gives a safe step size $1/L$ and a guaranteed decrease per step, the standard starting point of convergence proofs.

Definition 6.15: L-smoothness

f is L-smooth if $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$. For $f \in C^2$ this is equivalent to $-L\mathbf{I} \preceq \nabla^2 f \preceq L\mathbf{I}$ (all Hessian eigenvalues in $[-L, L]$).

Theorem 6.16: descent lemma

If f is L-smooth, $f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2}\|\mathbf{y} - \mathbf{x}\|^2$.

Proof. Subtract $\nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 \nabla f(\mathbf{x})^\top \mathbf{d}\, dt$ from the first Taylor formula, then use Cauchy–Schwarz and smoothness:

$$f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 (\nabla f(\mathbf{x} + t\mathbf{d}) - \nabla f(\mathbf{x}))^\top \mathbf{d}\, dt \leq \int_0^1 Lt\|\mathbf{d}\|^2 dt = \frac{L}{2}\|\mathbf{d}\|^2.$$

∎

Theorem 6.17: GD on smooth functions

GD satisfies $f(\mathbf{x}_{t+1}) \leq f(\mathbf{x}_t) - \eta(1 - \frac{L\eta}{2})\|\nabla f(\mathbf{x}_t)\|^2$, so it decreases f when $0 < \eta < 2/L$. With $\eta = 1/L$ and $f^\star = \inf f > -\infty$,

$$\min_{0 \leq t < T} \|\nabla f(\mathbf{x}_t)\|^2 \leq \frac{1}{T} \sum_{t<T} \|\nabla f(\mathbf{x}_t)\|^2 \leq \frac{2L\,(f(\mathbf{x}_0) - f^\star)}{T}.$$

Proof. Put $\mathbf{y} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$ in the descent lemma. With $\eta = 1/L$ the coefficient is $1/(2L)$; summing over t telescopes the left side to $f(\mathbf{x}_0) - f(\mathbf{x}_T) \leq f(\mathbf{x}_0) - f^\star$. ∎

This is a stationarity guarantee: for nonconvex f a small gradient may sit at a saddle point or a local minimum. For $f(x) = x^2/2$ ($L = 1$), $x_{t+1} = (1-\eta)x_t$ converges iff $0 < \eta < 2$; for $f = \frac{1}{2}\mathbf{x}^\top H\mathbf{x}$ each eigen-direction contracts by $1 - \eta\lambda_i$, so a small eigenvalue (poor conditioning) means slow progress while $\eta < 2/\lambda_{\max}$ is required for stability.

9 Stochastic Gradient Descent

Motivation. With noisy gradients the iterates no longer settle down at a fixed step size. The analysis shows how the noise floor depends on the step size and why decreasing step sizes are needed for convergence.

SGD uses $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\mathbf{g}_t$ with $\mathbb{E}_t\mathbf{g}_t = \nabla f(\mathbf{x}_t)$, where $\mathbb{E}_t$ conditions on the history up to $\mathbf{x}_t$.

Theorem 6.18: second moment is not variance

$\mathbb{E}_t\|\mathbf{g}_t\|^2 = \|\nabla f(\mathbf{x}_t)\|^2 + \mathbb{E}_t\|\mathbf{g}_t - \nabla f(\mathbf{x}_t)\|^2$.

Proof. Expand $\mathbf{g}_t = \nabla f + (\mathbf{g}_t - \nabla f)$; the cross term has conditional mean zero. ■

Theorem 6.19: $T^{-1/2}$ rate for SGD

If f is L-smooth, $\mathbb{E}_t\|\mathbf{g}_t\|^2 \le G$ and $\Delta = f(\mathbf{x}_0) - f^\star$, then for a constant step

$$\frac{1}{T}\sum_{t<T}\mathbb{E}\|\nabla f(\mathbf{x}_t)\|^2 \le \frac{\Delta}{\eta T} + \frac{L\eta G}{2}, \qquad \text{and with } \eta = \sqrt{\tfrac{2\Delta}{LGT}}: \quad \le \sqrt{\frac{2L\Delta G}{T}}.$$

Proof. Descent lemma: $f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta\nabla f(\mathbf{x}_t)^\top\mathbf{g}_t + \frac{L\eta^2}{2}\|\mathbf{g}_t\|^2$. Conditional expectation gives $\mathbb{E}_t f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta\|\nabla f(\mathbf{x}_t)\|^2 + \frac{L\eta^2}{2}G$. Take total expectation (tower property), sum over $t < T$ (telescoping), use $\mathbb{E}f(\mathbf{x}_T) \ge f^\star$ and divide by ηT. Finally $\frac{a}{\eta} + b\eta$ is minimized at $\eta = \sqrt{a/b}$ with value $2\sqrt{ab}$. ■

The noise term forces $\eta \sim T^{-1/2}$, slower than GD's T^{-1}. Since mini-batches of size B divide the noise variance by B, larger batches reduce G and permit larger steps.

10 Momentum

Motivation. Gradient descent zigzags in narrow valleys. Momentum averages past gradients so that oscillating components cancel and consistent ones accumulate; Nesterov's variant evaluates the gradient at a look-ahead point.

10.1 Heavy-ball momentum

Definition 6.20: momentum

$\mathbf{v}_{t+1} = \beta\mathbf{v}_t - \eta\nabla f(\mathbf{x}_t)$, $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}$, $\mathbf{v}_0 = \mathbf{0}$, $0 \le \beta < 1$.

Unrolling, $\mathbf{v}_{t+1} = -\eta\sum_{k=0}^{t}\beta^{t-k}\nabla f(\mathbf{x}_k)$: an exponentially weighted sum of past gradients (a velocity). For a constant gradient $\mathbf{g}$, $\|\mathbf{v}_t\| \to \eta\|\mathbf{g}\|/(1-\beta)$, an effective step $1/(1-\beta)$ times larger ($10\times$ for $\beta = 0.9$). Persistent directions accumulate; oscillating directions cancel, which reduces

zig-zagging in narrow valleys. Physically the gradient acts as a force and $1-\beta$ as friction: in the buffer form $\mathbf{m}_{t+1} = \mathbf{g}_t + \beta\mathbf{m}_t$ one has $\mathbf{m}_{t+1} - \mathbf{m}_t = \mathbf{g}_t - (1-\beta)\mathbf{m}_t$.

Theorem 6.21: momentum on a quadratic

For $f(x) = \frac{h}{2}x^2$, $x_{t+1} = (1+\beta-\eta h)x_t - \beta x_{t-1}$; the iteration converges iff both roots of $r^2 - (1+\beta-\eta h)r + \beta = 0$ lie in the open unit disk, which for $0 \le \beta < 1$ holds iff $0 < \eta h < 2(1+\beta)$. Complex roots (modulus $\sqrt{\beta}$) mean damped oscillation.

Proof. Substitute $\mathbf{v}_{t+1} = x_{t+1} - x_t$ and $\mathbf{v}_t = x_t - x_{t-1}$ into the velocity update. For the root condition use the Jury/Schur criterion for $r^2 + a_1 r + a_0$: $|a_0| < 1$, $1 + a_1 + a_0 > 0$, $1 - a_1 + a_0 > 0$, which here read $\beta < 1$, $\eta h > 0$, $\eta h < 2(1+\beta)$. ■

10.2 Nesterov momentum

Nesterov evaluates the gradient at the look-ahead point:

$$\mathbf{y}_t = \mathbf{x}_t + \beta\mathbf{v}_t, \qquad \mathbf{v}_{t+1} = \beta\mathbf{v}_t - \eta\nabla f(\mathbf{y}_t), \qquad \mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}.$$

Example 6.22: a numerical illustration

$f = x^2/2$, $x_0 = 1$, $\eta = 0.1$, $\beta = 0.9$. Both methods give $v_1 = -0.1$, $x_1 = 0.9$. Momentum: $v_2 = 0.9(-0.1) - 0.1(0.9) = -0.18$, $x_2 = 0.72$. Nesterov: $y_1 = 0.81$, $v_2 = -0.09 - 0.081 = -0.171$, $x_2 = 0.729$: it measures the slope where it is about to go and brakes earlier (not necessarily closer to the optimum at every step).

Theorem 6.23: two forms of Nesterov

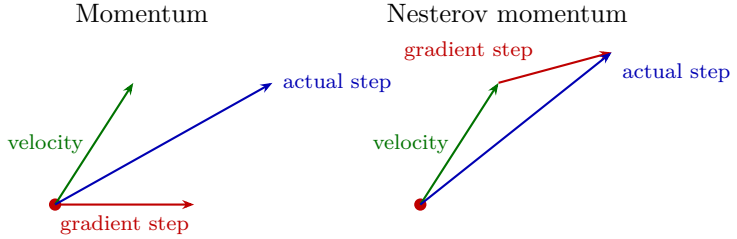
Form I: $\psi^{k+1} = \theta^k - \alpha g^k$, $\theta^{k+1} = \psi^{k+1} + \beta(\psi^{k+1} - \psi^k)$, $\psi^0 = \theta^0$. Form II (PyTorch `nesterov=True`): $m^{k+1} = g^k + \beta m^k$, $v^{k+1} = \beta m^{k+1} + g^k$, $\theta^{k+1} = \theta^k - \alpha v^{k+1}$, $m^0 = 0$. Here g^k is evaluated at θ^k. For the same θ^0 and $g^0, g^1, \ldots$ both produce the same $\theta^1, \theta^2, \ldots$

Proof. *Both satisfy one recursion.* Form I with $\psi^{k+1} = \theta^k - \alpha g^k$ and $\psi^k = \theta^{k-1} - \alpha g^{k-1}$ gives, for $k \ge 1$,

$$\theta^{k+1} = \theta^k - \alpha g^k + \beta(\theta^k - \theta^{k-1}) - \alpha\beta(g^k - g^{k-1}).$$

Form II: $\theta^{k+1} = \theta^k - \alpha g^k - \beta(\alpha g^k + \alpha\beta m^k)$, and from the previous step $\alpha\beta m^k = \alpha v^k - \alpha g^{k-1} = \theta^{k-1} - \theta^k - \alpha g^{k-1}$; substituting gives the same recursion. Both start with $\theta^1 = \theta^0 - \alpha(1+\beta)g^0$. A second-order recursion with the same two initial values has the same solution (induction). *Alternative.* Define $\psi^k := \theta^k + \alpha\beta m^k$ in Form II. Then $\psi^0 = \theta^0$, $\psi^{k+1} = \theta^k - \alpha g^k$ and $\psi^{k+1} - \psi^k = -\alpha m^{k+1}$, from which Form I's second equation follows. ■

Interpretation: ψ is the actual position (often written x_t), θ the look-ahead point $x_t + \beta v_t$ at which gradients are taken and which PyTorch stores as the parameter; m is the velocity of ψ scaled by $-1/\alpha$. Both converge to the same limit since $\theta^k - \psi^k = \beta(\psi^k - \psi^{k-1}) \to 0$.



Momentum adds the gradient step taken at the current point to the velocity; Nesterov evaluates the gradient after first moving along the velocity.

11 Adaptive Methods

Motivation. Different parameters often need different step sizes. AdaGrad, RMSProp and Adam rescale each coordinate by a running estimate of its gradient magnitude; this section derives their update rules and the bias correction in Adam.

All operations below are coordinatewise.

Definition 6.24: AdaGrad, RMSProp, Adam

AdaGrad: $\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t$, $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\, \mathbf{g}_t \oslash (\sqrt{\mathbf{r}_t} + \epsilon)$.
RMSProp: $\mathbf{r}_t = \rho \mathbf{r}_{t-1} + (1-\rho)\mathbf{g}_t \odot \mathbf{g}_t$, same update.
Adam: $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$, $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t \odot \mathbf{g}_t$, $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$, $\hat{\mathbf{v}}_t = \mathbf{v}_t/(1-\beta_2^t)$, $\mathbf{x}_t = \mathbf{x}_{t-1} - \eta\, \hat{\mathbf{m}}_t \oslash (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$. Typical $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$.

Remark 6.25: what each piece does

AdaGrad's accumulator grows forever, so its effective rate $\eta/\sqrt{r_t}$ shrinks (for a constant gradient g the step is $\eta/\sqrt{t}$). RMSProp forgets old gradients; for constant $g = 2$, $\rho = 0.9$, $\eta = 0.1$, $r_1 = 0.4$, $r_2 = 0.76$, steps $0.316, 0.229$, tending to 0.1. Adam = momentum's numerator (direction) + RMSProp's denominator $\sqrt{\mathbf{v}}$ (per-coordinate scale) + bias correction. (Nesterov plus Adam is NAdam.)

11.1 Exponential moving averages and bias correction

Theorem 6.26: EMA of iid gradients

If $g_0, g_1, \dots$ are iid with mean μ and variance s^2, and $m_{k+1} = \beta m_k + (1-\beta) g_k$, $m_0 = 0$, then

$$\mathbb{E} m_k = (1-\beta^k)\mu, \qquad \operatorname{Var} m_k = s^2 \frac{1-\beta}{1+\beta}(1-\beta^{2k}),$$

with limits μ and $s^2 \frac{1-\beta}{1+\beta}$. The corrected $\hat{m}_k = m_k/(1-\beta^k)$ is unbiased with variance $s^2 \frac{(1-\beta)(1+\beta^k)}{(1+\beta)(1-\beta^k)}$ (equal to s^2 at $k = 1$).

Proof. m_k depends only on $g_0, \dots, g_{k-1}$, so it is independent of g_k. Mean by induction: $\beta(1-\beta^k)\mu + (1-\beta)\mu = (1-\beta^{k+1})\mu$. Variance: $\operatorname{Var} m_{k+1} = \beta^2 \operatorname{Var} m_k + (1-\beta)^2 s^2$; writing $(1-\beta)^2 = \frac{1-\beta}{1+\beta}(1-\beta^2)$, the induction step is $\beta^2(1-\beta^{2k}) + (1-\beta^2) = 1 - \beta^{2k+2}$. Correction: divide by $(1-\beta^k)^2$ and use $1-\beta^{2k} = (1-\beta^k)(1+\beta^k)$. ∎

Without correction Adam's first step is $\frac{1-\beta_1}{\sqrt{1-\beta_2}} \operatorname{sign}(g_1)\eta \approx 3.16\,\eta$; with correction $\hat{m}_1 = g_1$, $\hat{v}_1 = g_1^2$ and the step is exactly $\eta \operatorname{sign}(g_1)$. The correction removes the zero-initialization bias at the price of a larger early variance.

11.2 Special cases and invariances

Theorem 6.27: Adam contains RMSProp and signSGD

For Adam without bias correction: $\beta_1 = 0$ gives RMSProp with $\rho = \beta_2$; $\beta_1 = \beta_2 = 0$, $\epsilon = 0$ give signSGD $\theta_{k+1} = \theta_k - \eta\ \mathrm{sign}(g_k)$ (if no coordinate of g_k is 0). With $\epsilon = 0$, Adam's iterates are unchanged when the loss is multiplied by $c > 0$, while SGD's step scales by c.

Proof. $\beta_1 = 0$ makes $m_{k+1} = g_k$. With $\beta_2 = 0$ also, $\sqrt{v_{k+1}} = |g_k|$. Scaling $g \mapsto cg$ scales m by c and $\sqrt{v}$ by c. ■

11.3 Weight decay is not ℓ_2 regularization

For plain SGD, ℓ_2 regularization $\theta - \eta(g + \lambda\theta) = (1 - \eta\lambda)\theta - \eta g$ is the same as weight decay. With momentum it is not:

Theorem 6.28: no matching exists

SGD with momentum $v_{k+1} = g_k + \beta v_k$, $\theta_{k+1} = \theta_k - \alpha v_{k+1}$ and $\alpha, \beta > 0$. Weight decay: $\theta_{k+1} = \theta_k - \alpha v_{k+1} - \mu\theta_k$. ℓ_2: $v_{k+1} = (g_k + \lambda\theta_k) + \beta v_k$. No choice of $\mu, \lambda > 0$ makes the two methods equivalent.

Proof. Take $g_k \equiv 0$, $v_0 = 0$, $\theta_0 \neq 0$. Weight decay: $v_k = 0$ and $\theta_k = (1 - \mu)^k\theta_0$. ℓ_2: $v_1 = \lambda\theta_0$, $\theta_1 = (1 - \alpha\lambda)\theta_0$, $v_2 = \lambda(1 - \alpha\lambda + \beta)\theta_0$, $\theta_2 = [(1 - \alpha\lambda)^2 - \alpha\beta\lambda]\theta_0$. Matching θ_1 forces $\mu = \alpha\lambda$, and then matching θ_2 forces $\alpha\beta\lambda = 0$, a contradiction. (Indeed the ℓ_2 sequence is geometric for no ratio.) ■

In Adam the ℓ_2 gradient $\lambda\theta$ is divided by its own RMS, so every weight moves by about η regardless of its size ($\theta_1 = \theta_0 - \eta\ \mathrm{sign}(\theta_0)$ when $g \equiv 0$), losing proportional shrinkage; AdamW applies the decay $(1 - \eta\lambda)\theta$ outside the adaptive scaling.

Chapter 7. Convolutional Neural Networks and Architectures

Convolution, stride, padding, pooling, equivariance; AlexNet, LRN, ZFNet, VGG, GoogLeNet/Inception, ResNet, saliency maps

1 Why a New Layer

An image $X \in \mathbb{R}^{H\times W\times C}$ fed to a fully connected layer is flattened, and $z = A\mathbf{x} + \mathbf{b}$ with $A \in \mathbb{R}^{M\times HWC}$ has $M(HWC+1)$ parameters: for $32\times 32\times 3$ input and $M = 1000$, already 3,073,000. Flattening loses no numbers, but an unrestricted matrix does not encode which pixels are neighbors or that a pattern can appear anywhere. Two assumptions about images motivate convolution:

- **Locality**: nearby pixels jointly form edges, corners and textures, so a unit can look at a small patch (local connectivity).
- **Stationarity**: the same pattern may occur at any position, so the same detector should be used everywhere (weight sharing).

Definition 7.1: equivariance and invariance

Let $T_{\boldsymbol{\tau}}$ shift an image, $(T_{\boldsymbol{\tau}}X)_{i,j,c} = X_{i-\tau_1, j-\tau_2, c}$. A feature extractor F is shift *equivariant* if $F(T_{\boldsymbol{\tau}}X) = T_{\boldsymbol{\tau}}F(X)$ (the output moves with the input) and a classifier G is shift *invariant* if $G(T_{\boldsymbol{\tau}}X) = G(X)$ (the output does not change). More generally, for a group acting by $\rho_{\text{in}}(g)$, $\rho_{\text{out}}(g)$, f is equivariant if $f(\rho_{\text{in}}(g)x) = \rho_{\text{out}}(g)f(x)$; invariance is the case $\rho_{\text{out}} = \text{id}$.

2 Convolution as a Local Dot Product

2.1 A fixed filter: moving average

With the 3×3 averaging kernel (1/9 everywhere), $Y_{i,j} = \frac{1}{9}\sum_{u,v=0}^{2} X_{i+u,j+v}$: on an image of 0s and 90s a window with r bright pixels outputs $10r$. Averaging suppresses isolated pixels but blurs edges; a 5×5 average smooths more. A CNN *learns* the kernel; learned weights may be negative and need not sum to one.

2.2 Single and multiple channels

Definition 7.2: convolutional layer

With D filters $W \in \mathbb{R}^{D\times K\times K\times C}$, biases $\mathbf{b} \in \mathbb{R}^{D}$, stride S, padding P,

$$Z_{i,j,d} = b_d + \sum_{u=0}^{K-1}\sum_{v=0}^{K-1}\sum_{c=1}^{C} W_{d,u,v,c}\,\tilde{X}_{Si+u,\,Sj+v,\,c},$$

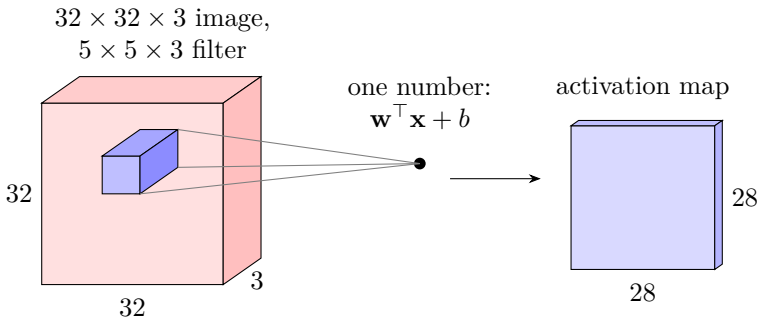
where $\tilde{X}$ is the padded input. Each output is a dot product of a $K\times K\times C$ patch with a filter spanning *all* input channels, plus a bias. The array $Z_{:,:,d}$ is the feature map of filter d; stacking the D maps gives D output channels. This operation is cross-correlation (no kernel flip); since kernels are learned, the distinction does not matter for expressiveness.

Example 7.3: one channel, 2×2 filter

$X = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{pmatrix}$, $W = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}$, $b = 1$, $S = 1$, $P = 0$: $Z = \begin{pmatrix} 4 & 8 \\ 1 & 1 \end{pmatrix}$. The same four weights and bias are reused at the four positions.

Example 7.4: a 5×5 example

With a 5×5 image and $W = \begin{pmatrix} 1 & 2 & 3 \\ -4 & 7 & 4 \\ 2 & -5 & 1 \end{pmatrix}$ ($b = 0$): unpadded $Z = \begin{pmatrix} 51 & 66 & 20 \\ 31 & 49 & 101 \\ 15 & 53 & -2 \end{pmatrix}$; with $P = 1$ the output is 5×5 and its central 3×3 block equals the unpadded output (those windows contain the same pixels).



A filter spans the full depth of the input; sliding it over all spatial positions gives one 28×28 activation map.

2.3 Parameter counts

One $5 \times 5 \times 3$ filter has 75 weights and one bias; six filters on $32 \times 32 \times 3$ give a $28 \times 28 \times 6$ output with $6(75 + 1) = 456$ parameters. For the same input and output shape:

Layer	Parameters
Fully connected	$4704(3072 + 1) = 14{,}455{,}392$
Locally connected, unshared	$28^2 \cdot 6(75 + 1) = 357{,}504$
Convolution (shared)	$6(75 + 1) = 456$

Locality removes long-range connections; sharing removes independent copies. As a matrix acting on the flattened input, convolution is sparse with repeated entries (a 1D length-2 filter (a, b) on $(x_1, \ldots, x_4)$ gives a banded matrix with only two free numbers).

3 Stride, Padding and Output Size

Motivation. Designing a CNN requires knowing the shape of every layer's output. A single formula relates input size, kernel size, padding and stride.

Theorem 7.5: output size

For input length N, kernel K, stride S, padding P (symmetric), the number of output positions is

$$N' = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1, \qquad C' = D.$$

Proof. The padded length is $N + 2P$. A window starting at tS ends at $tS + K - 1$, which must be $\leq N + 2P - 1$, i.e. $t \leq (N + 2P - K)/S$. The admissible integers are $t = 0, \ldots, \lfloor (N + 2P - K)/S \rfloor$. ■

$N = 7$, $K = 3$: $S = 1 \Rightarrow 5$, $S = 2 \Rightarrow 3$, $S = 3 \Rightarrow 2$ (the last does not "fit" exactly; a border is left unused). For $S = 1$ and odd K, $P = (K - 1)/2$ preserves the size (padding $1, 2, 3$ for $K = 3, 5, 7$). Zero padding is an explicit boundary assumption, not recovered information.

Example 7.6: further examples

Input $52 \times 52 \times 128$, $D = 256$.
(1) $K = 3, S = 1, P = 1$: output $52 \times 52 \times 256$, parameters $256(9 \cdot 128 + 1) = 295{,}168$; each filter is $3 \times 3 \times 128$.
(2) $K = 4, S = 2, P = 1$: $\lfloor (52 + 2 - 4)/2 \rfloor + 1 = 26$, output $26 \times 26 \times 256$, parameters $256(16 \cdot 128 + 1) = 524{,}544$. Parameters, activations (HWD) and multiply–accumulates ($\approx HWDK^2C$) are different costs.

Stacking: $32 \times 32 \times 3 \to 28 \times 28 \times 6 \to 24 \times 24 \times 10$ with ten $5 \times 5 \times 6$ filters ($10(150 + 1) = 1510$ parameters). Two stacked stride-1 5×5 layers see a 9×9 patch (receptive field $5 + 4$). With strides, $r_\ell = r_{\ell-1} + (K_\ell - 1) j_{\ell-1}$ and $j_\ell = S_\ell j_{\ell-1}$, starting from $r_0 = j_0 = 1$.

4 Learning Filters

For an unpadded stride-1 layer with upstream $G_{i,j,d} = \partial L / \partial Z_{i,j,d}$,

$$\frac{\partial L}{\partial W_{d,u,v,c}} = \sum_{i,j} G_{i,j,d} X_{i+u,j+v,c}, \qquad \frac{\partial L}{\partial b_d} = \sum_{i,j} G_{i,j,d} :$$

every position contributes to the update of the shared weight (the backward counterpart of weight sharing).

5 Shift Equivariance

Motivation. Shifting an image should shift the features, not change them. Convolution has exactly this property, which is the mathematical reason CNNs need far fewer parameters than fully connected networks on images.

Theorem 7.7: convolution commutes with shifts

On an infinite grid (or with circular indices), stride-1 convolution satisfies $C_W(T_{\boldsymbol{\tau}} X) = T_{\boldsymbol{\tau}} C_W(X)$, and pointwise nonlinearities also commute with shifts. Global average pooling of a circularly shifted map is unchanged, so GAP $\circ$ (convolutions, ReLUs) is shift invariant.

Proof. $[C_W(T_{\boldsymbol{\tau}} X)]_{i,j,d} = b_d + \sum_{u,v,c} W_{d,u,v,c} X_{i+u-\tau_1, j+v-\tau_2, c} = [C_W X]_{i-\tau_1, j-\tau_2, d}$; the coefficients do not depend on the absolute position (i, j). A circular shift permutes positions, and an average is invariant under permutations. ■

Remark 7.8: when exactness fails

With zero padding, content shifted across the border is lost and zeros enter, so equality holds only away from the boundary. A stride-S layer (or pooling) is compatible only with shifts that are multiples of S: for width-2, stride-2 max pooling M, $M(T_2x) = T_1M(x)$ but $M(T_1x)$ need not be a shift of $M(x)$ (e.g. $x = (0, 1, 2, 0)$, $M(x) = (1, 2)$, $M(T_1x) = (0, 2)$).

6 1×1 Convolution, Pooling and Global Average Pooling

1×1 convolution. At each position $\mathbf{y}_{i,j} = A\mathbf{x}_{i,j} + \mathbf{b}$ with $A \in \mathbb{R}^{D\times C}$ shared across positions: it mixes channels, not neighbors. $56 \times 56 \times 64 \to 56 \times 56 \times 32$ costs $32(64 + 1) = 2080$ parameters.

Pooling. Channelwise, no parameters: $Y^{\max}_{i,j,c} = \max_{u,v} X_{iS+u,jS+v,c}$ or the average. For $X = \begin{pmatrix} 5 & 10 & 6 & 7 \\ 4 & 0 & 4 & 3 \\ 2 & 4 & 3 & 4 \\ 0 & 8 & 1 & 6 \end{pmatrix}$ and 2×2, stride 2: average $\begin{pmatrix} 4.75 & 5 \\ 3.5 & 3.5 \end{pmatrix}$, max $\begin{pmatrix} 10 & 7 \\ 8 & 6 \end{pmatrix}$. Pooling reduces resolution and gives limited robustness to small shifts within a window. Backward: max pooling routes the gradient to the maximizer; average pooling spreads g/K^2.

Global average pooling. $g_c = \frac{1}{HW}\sum_{i,j} X_{i,j,c}$ maps $H \times W \times C \to C$; a classifier on top needs $Q(C + 1)$ instead of $Q(HWC + 1)$ parameters. It discards spatial arrangement entirely.

7 Algebraic Facts about CNN Layers

Motivation. The building blocks of CNNs (convolution, ReLU, pooling) satisfy simple algebraic identities. They tell us when layers can be reordered or merged without changing the network.

Theorem 7.9: positive homogeneity

A network built only from bias-free convolutions and linear layers, ReLU, max pooling, reshaping and dropout satisfies $f(\lambda X) = \lambda f(X)$ for all $\lambda \geq 0$.

Proof. Each component is positively homogeneous: linear maps for all λ; $\max(0, \lambda t) = \lambda \max(0, t)$ and $\max_i \lambda x_i = \lambda \max_i x_i$ for $\lambda \geq 0$ (scaling by $\lambda > 0$ preserves order and signs; $\lambda = 0$ is trivial); dropout multiplies by a fixed mask. Compositions of positively homogeneous maps are positively homogeneous: $g_1(g_2(\lambda X)) = g_1(\lambda g_2(X)) = \lambda g_1(g_2(X))$. Biases break the property, e.g. $\mathrm{ReLU}(x+1)$ at $x = 0$. ■

Theorem 7.10: conv–MP–ReLU orders coincide

conv→MaxPool→ReLU, conv→ReLU→MaxPool and conv→ReLU→MaxPool→ReLU are the same function.

Proof. For nondecreasing ϕ, $\max_i \phi(x_i) = \phi(\max_i x_i)$: $\phi(x_i) \leq \phi(\max x)$ for all i with equality at the maximizer. ReLU is nondecreasing and elementwise, and max pooling is a windowwise max, so $\mathrm{ReLU} \circ \mathrm{MP} = \mathrm{MP} \circ \mathrm{ReLU}$; finally $\mathrm{ReLU} \circ \mathrm{ReLU} = \mathrm{ReLU}$. (For average pooling only avg ReLU $\geq$ ReLU avg holds, by convexity.) ■

Theorem 7.11: convolutions that implement given maps

With PyTorch's convention $Y_{o,i,j} = b_o + \sum_c \sum_{k,l=1}^{3} w_{o,c,k,l} \tilde{X}_{c,i+k-2,j+l-2}$ (3×3, padding 1), reading the value at offset $(\Delta i, \Delta j)$ requires a 1 at $(k,l) = (2+\Delta i, 2+\Delta j)$.

- Identity $C \to C$: $w_{c,c',k,l} = \delta_{cc'}\delta_{k2}\delta_{l2}$, $\mathbf{b} = \mathbf{0}$.
- Colorwise finite differences $Y_{2c-1} = X_{c,i+1,j} - X_{c,i,j}$, $Y_{2c} = X_{c,i,j+1} - X_{c,i,j}$: $w_{2c-1,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, $w_{2c,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$, all other slices zero; zero padding supplies the out-of-range zeros.

8 Vocabulary and Costs

A network maps an image to class scores and then probabilities, $X \mapsto h(X) \mapsto \mathbf{s}(X) \in \mathbb{R}^Q \mapsto \mathbf{p}(X)$. For a convolution with D filters $K \times K \times C$ producing $H' \times W' \times D$:

$$N_{\text{param}} = D(K^2C + 1), \qquad N_{\text{act}} = H'W'D, \qquad N_{\text{MAC}} \approx H'W'DK^2C.$$

An FC layer $\mathbb{R}^n \to \mathbb{R}^m$ has $m(n+1)$ parameters. Top-1 error asks whether the highest score is correct; top-5 error asks whether the true label is missing from the five highest scores.

9 AlexNet

Five convolutional and three FC layers (eight weight layers):

Stage	Settings	Output
Input	RGB	$227 \times 227 \times 3$
Conv1	96, 11×11, $S = 4$, $P = 0$	$55 \times 55 \times 96$ $((227-11)/4+1)$
Pool1, Norm1	3×3, $S = 2$; LRN	$27 \times 27 \times 96$
Conv2	256, 5×5, $P = 2$	$27 \times 27 \times 256$
Pool2, Norm2	3×3, $S = 2$; LRN	$13 \times 13 \times 256$
Conv3, Conv4	384, 3×3, $P = 1$	$13 \times 13 \times 384$
Conv5	256, 3×3, $P = 1$	$13 \times 13 \times 256$
Pool3	3×3, $S = 2$	$6 \times 6 \times 256 = 9216$
FC6, FC7, FC8	$9216 \to 4096 \to 4096 \to 1000$	1000 scores

The FC layers dominate: $4096 \cdot 9217 = 37{,}752{,}832$, $4096 \cdot 4097 = 16{,}781{,}312$, $1000 \cdot 4097 = 4{,}097{,}000$, together 58,631,144 parameters, whereas Conv1 has $96(11^2 \cdot 3 + 1) = 34{,}944$. Design choices:

- **ReLU**: avoids sigmoid/tanh saturation on the positive side.
- **Large first stride**: $227 \to 55$ immediately reduces memory and computation (at the cost of detail).
- **Local response normalization**: $a_{i,j,c} \mapsto a_{i,j,c}/(k + \alpha \sum_{r \in N(c)} a_{i,j,r}^2)^\beta$ normalizes across neighboring channels (not batch normalization).
- **Dropout** 0.5 in the FC layers against overfitting.

10 ZFNet and VGG

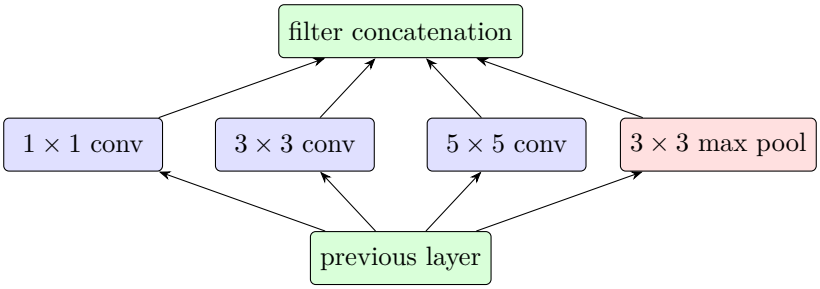
ZFNet tunes AlexNet's kernel sizes and strides. VGG uses deeper stacks of small 3×3 convolutions: two stride-1 3×3 layers have a 5×5 receptive field with $18C^2$ weights (versus $25C^2$), three have 7×7 with $27C^2$ (versus $49C^2$), and interleaved ReLUs add nonlinearity. Receptive fields obey $r_\ell = r_{\ell-1} + (K_\ell - 1)j_{\ell-1}$, $j_\ell = S_\ell j_{\ell-1}$.

11 GoogLeNet and the Inception Module

Motivation. Which filter size should a layer use? Inception uses several in parallel and concatenates the results, with 1×1 convolutions to keep the cost under control.

11.1 Naive module

For input $28\times28\times256$, parallel branches with padding chosen to keep 28×28: 1×1 (128), 3×3 (192), 5×5 (96), 3×3 max pool (stride 1). Channel concatenation gives $128+192+96+256=672$ channels. Weights: $256\cdot128+9\cdot256\cdot192+25\cdot256\cdot96=1{,}089{,}536$.



The naive Inception module: parallel convolutions of several sizes and a pooling branch, concatenated along the depth.

11.2 Bottleneck module

Insert 1×1 reductions to 64 channels before the 3×3 and 5×5 convolutions and after pooling: outputs $128+192+96+64=480$ channels and

$$32{,}768+(16{,}384+110{,}592)+(16{,}384+153{,}600)+16{,}384=346{,}112 \text{ weights}$$

(about 330k), a 68% reduction.

Theorem 7.12: when a bottleneck saves weights

A direct $K\times K$ map $C\to D$ costs K^2CD; reducing to R channels first costs $CR+K^2RD$. This is cheaper iff $R<\dfrac{K^2CD}{C+K^2D}$ (for $K=5, C=256, D=96$ the threshold is ≈ 231).

The 1×1 convolution is a channel-mixing matrix applied at every position (see above). GoogLeNet stacks such modules (22 weight layers) and replaces the large FC stack by global average pooling plus a final classifier.

11.3 Split–transform–merge

A $256 \to 128 \to 128 \to 256$ conv–ReLU block (1×1, 3×3 with padding 1, 1×1, all with biases) has $32{,}896 + 147{,}584 + 33{,}024 = 213{,}504$ parameters. Thirty-two parallel $256 \to 4 \to 4 \to 256$ paths summed at the end have $32(1028 + 148 + 1280) = 78{,}592$ parameters; the 3×3 cost drops by the group factor 32. Implementation: three `nn.ModuleLists` of 32 convolutions, ReLU inside each path, then a sum (equivalently grouped convolutions with `groups=32`).

12 Network in Network: Exact versus Approximate Invariance

NiN stacks mlpconv blocks (a spatial convolution followed by 1×1 convolutions, ReLUs, 3×3 stride-2 max pooling with ceil mode, dropout) and ends with average pooling over the final $8 \times 8 \times 10$ map. Take X with a single 1 at $(j, k) = (19, 19)$ and $\tilde{X}$ the same shifted to $(23, 19)$ (0-based, no biases, eval mode). Tracking where outputs can be nonzero: bias-free layers map zeros to zeros; a $K \times K$ convolution with padding $(K-1)/2$ widens the support by $(K-1)/2$; 1×1 convolutions and ReLU keep it; pool output p sees rows $2p, 2p+1, 2p+2$.

	rows for X	rows for $\tilde{X}$
conv1 (5×5)	17–21	21–25
out1 (16×16)	8–10	10–12
conv2 (5×5)	6–12	8–14
out2 (8×8)	2–6	3–7
out3 (3×3 conv)	1–7	2–8, truncated to 2–7

Columns stay 8–10, 2–6, 1–7. The shift 4 is a multiple of the total stride 4, so out1 is exactly shifted by 2 and out2 by 1; then out3 of $\tilde{X}$ equals out3 of X shifted down one row, except that row 7 of X's map would land on row 8, outside the 8×8 map. Hence the averaged outputs differ exactly by $\frac{1}{64}\sum_k \text{out3}(X)_{c,7,k}$: approximately equal, but not exactly equal in general.

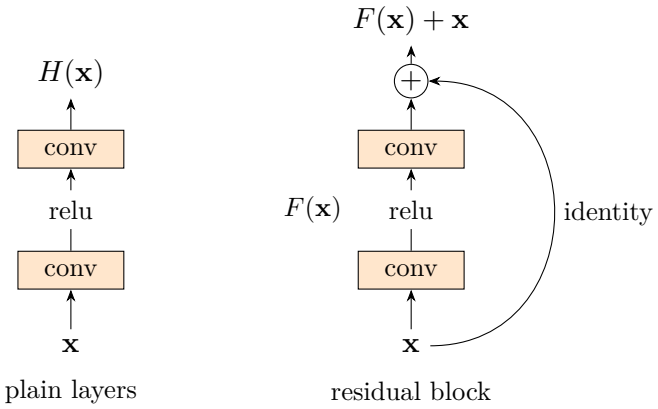
13 Why Depth Can Fail, and Residual Networks

Deeper plain networks can have *higher training error*: this is not overfitting but an optimization (degradation) problem. A deeper network that can represent identity layers contains the shallower one, so its *optimal* training loss is not worse, yet gradient methods may not find it. For $\mathbf{x}_{\ell+1} = f_\ell(\mathbf{x}_\ell)$, $\nabla_{\mathbf{x}_0} L = J_0^\top \cdots J_{L-1}^\top \nabla_{\mathbf{x}_L} L$; if every Jacobian has norm at most $\rho < 1$ the gradient shrinks like ρ^L ($0.9^{50} \approx 0.005$).

Theorem 7.13: residual block

$\mathbf{y} = \mathbf{x} + F(\mathbf{x}; \boldsymbol{\theta})$ gives $\dfrac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + J_F(\mathbf{x})$ and $\nabla_{\mathbf{x}} L = \mathbf{g} + J_F^\top \mathbf{g}$ with $\mathbf{g} = \nabla_{\mathbf{y}} L$.

The identity path carries the gradient even when J_F is small, and the block can represent the identity by $F = 0$: the network learns a residual $F = H - \text{id}$ rather than the whole map H. This is a reparametrization, not a guarantee (e.g. $J_F = -\mathbf{I}$ cancels the gradient; a ReLU after the sum alters the identity). If shapes differ, use a projection shortcut $\mathbf{y} = P(\mathbf{x}) + F(\mathbf{x})$ (e.g. a 1×1 or strided convolution). Unlike Inception's concatenation, residual addition keeps the channel count. A bottleneck block $256 \to 64 \to 64 \to 256$ (1×1, 3×3, 1×1) has $2 \cdot 256 \cdot 64 + 9 \cdot 64^2 = 69{,}632$ weights versus $18 \cdot 256^2 = 1{,}179{,}648$ for two full-width 3×3 layers. In the CIFAR-10 plots, residual networks remove the training-error degradation seen in plain networks. The full backpropagation formulas for a residual MLP are derived in Chapter 5.



A residual block learns $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$; with $F = 0$ it is the identity, and gradients pass through the shortcut.

14 Saliency Maps

To see which pixels a prediction is sensitive to, differentiate the selected class score with respect to the input (not the weights):

$$G_{i,j,k} = \frac{\partial s_c(X)}{\partial X_{i,j,k}}, \qquad M_{i,j} = \max_k |G_{i,j,k}|.$$

By Taylor's theorem $s_c(X + \Delta X) \approx s_c(X) + \langle \nabla_X s_c(X), \Delta X \rangle$: a large $|G|$ means a small change of that pixel changes the score a lot, and the sign says in which direction. For $s(\mathbf{x}) = 2x_1 - 3x_2 + 0.1x_3$, the gradient $(2, -3, 0.1)$ ranks x_2 first. Saliency is local sensitivity, not a complete causal explanation.

Chapter 8. Universal Approximation, Barron's Theorem and Generalization

Lectures 10/21, 10/26: measures, normed spaces, Cybenko's theorem; Barron's theorem and proof; Rademacher complexity

1 Three Sources of Error

Let f^* be the ideal target, $\mathcal{F}$ the class of functions a given architecture can represent, f_{app} the best element of $\mathcal{F}$, f_{gen} the best element on the finite training sample (empirical risk minimizer) and f_{opt} what the optimizer actually returns. The total error splits as

$$\underbrace{f^* \to f_{\text{app}}}_{\text{approximation}} + \underbrace{f_{\text{app}} \to f_{\text{gen}}}_{\text{generalization (estimation)}} + \underbrace{f_{\text{gen}} \to f_{\text{opt}}}_{\text{optimization}}.$$

Chapter 6 dealt with optimization; this chapter shows that the approximation error of one-hidden-layer networks can be made arbitrarily small (universality) and, for Barron functions, quantifies it.

2 Preliminaries from Analysis

2.1 Measures

Definition 8.1: measures on a compact set $K \subset \mathbb{R}^n$

The **Borel sets** of K form the smallest collection containing all open subsets of K and closed under complements in K and countable unions. A (nonnegative) **measure** μ assigns $\mu(A) \in [0, \infty]$ to Borel sets with

$$\mu(\emptyset) = 0, \qquad \mu\Big(\bigcup_{j=1}^{\infty} A_j\Big) = \sum_{j=1}^{\infty} \mu(A_j) \quad \text{for pairwise disjoint } A_j.$$

A **signed measure** is the same with real (possibly negative) values, e.g. a difference of two finite measures. $\mu = 0$ means $\mu(A) = 0$ for every Borel set A. $M(I_n)$ denotes the finite signed regular Borel measures on the unit cube $I_n = [0, 1]^n$.

2.2 Normed spaces and density

Definition 8.2: linear and normed spaces

A real **linear space** is a set closed under addition and multiplication by reals obeying the usual vector-space rules; e.g. $C(K) = \{f : K \to \mathbb{R} \text{ continuous}\}$, since $af + bg \in C(K)$ with $(af + bg)(x) = af(x) + bg(x)$. A **normed linear space** is a linear space X with $\|\cdot\| : X \to [0, \infty)$ such that

$$\|u\| = 0 \iff u = 0, \qquad \|cu\| = |c|\|u\|, \qquad \|u + v\| \le \|u\| + \|v\|.$$

On $C(K)$ we use the sup norm $\|f\|_\infty = \max_{x \in K} |f(x)|$. A subset $D \subseteq X$ is **dense** in X if for every $f \in X$ and $\varepsilon > 0$ there is $g \in D$ with $\|f - g\| < \varepsilon$.

Density in $(C(I_n), \|\cdot\|_\infty)$ is exactly uniform approximation: every continuous function can be approximated within ε at every point simultaneously.

3 Cybenko's Universal Approximation Theorem

Motivation. Can neural networks represent the functions we want to learn? Cybenko's theorem says yes: one hidden layer with a sigmoidal activation can approximate any continuous function on a compact set uniformly. The proof is an existence argument from functional analysis (Hahn–Banach and Riesz).

Definition 8.3: sigmoidal and discriminatory

$\sigma : \mathbb{R} \to [0,1]$ is **sigmoidal** if $\lim_{t\to-\infty} \sigma(t) = 0$ and $\lim_{t\to+\infty} \sigma(t) = 1$. For $\mu \in M(I_n)$, σ is **discriminatory** if

$$\int_{I_n} \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu(\mathbf{x}) = 0 \quad \text{for all } \mathbf{w} \in \mathbb{R}^n,\ \theta \in \mathbb{R} \qquad \Longrightarrow \qquad \mu = 0.$$

Theorem 8.4: Cybenko 1989 (UAT)

Let σ be any continuous sigmoidal function. Then finite sums

$$G(\mathbf{x}) = \sum_{j=1}^{N} \alpha_j\, \sigma(\mathbf{w}_j^\top \mathbf{x} + \theta_j), \qquad \mathbf{w}_j \in \mathbb{R}^n,\ \alpha_j, \theta_j \in \mathbb{R},\ N \in \mathbb{N},$$

are dense in $C(I_n)$: for every continuous f and $\varepsilon > 0$ some G satisfies $\sup_{\mathbf{x}\in I_n} |f(\mathbf{x}) - G(\mathbf{x})| < \varepsilon$.

The proof combines three ingredients: a Hahn–Banach consequence (Theorem 8.5), the Riesz representation theorem (Theorem 8.6), and the discriminatory property (Theorem 8.9).

3.1 The functional-analytic lemmas

Theorem 8.5: a consequence of Hahn–Banach

Let W be a linear, non-dense subspace of a normed linear space X. Then there is a bounded linear functional L on X with $L \neq 0$ on X and $L|_W = 0$.

Proof. The closure $\overline{W}$ is a closed subspace and $\overline{W} \neq X$, so pick $x_0 \notin \overline{W}$; then $d = \inf_{w\in\overline{W}} \|x_0 - w\| > 0$. On $\overline{W} \oplus \mathbb{R}x_0$ define $\ell(w + tx_0) = t$. It is linear, vanishes on W, $\ell(x_0) = 1$, and bounded: $|t|d \le |t|\|x_0 + w/t\| = \|w + tx_0\|$ for $t \neq 0$, so $\|\ell\| \le 1/d$. The Hahn–Banach theorem extends ℓ to X with the same norm. ∎

Theorem 8.6: Riesz representation theorem

Let $K \subset \mathbb{R}^n$ be compact. For every bounded linear functional F on $C(K)$ there is a unique finite signed (regular Borel) measure μ on K with

$$F(f) = \int_K f(\mathbf{x})\, d\mu(\mathbf{x}) \qquad \text{for all } f \in C(K).$$

(For positive functionals the measure is nonnegative.)

Theorem 8.7: an annihilating measure

Let W be a linear, non-dense subspace of $C(I_n)$. Then there is $\mu \in M(I_n)$, $\mu \neq 0$, with $\int_{I_n} h\, d\mu = 0$ for all $h \in W$.

Proof [Supplement]. Theorem 8.5 with $X = C(I_n)$ gives a bounded linear $L \neq 0$ with $L|_W = 0$. Riesz represents $L(f) = \int f\, d\mu$; $\mu \neq 0$ because $L \neq 0$; and for $h \in W$, $\int h\, d\mu = L(h) = 0$. ■

3.2 Density for discriminatory functions

Theorem 8.8: density for discriminatory activations

If σ is continuous and discriminatory, the sums G above are dense in $C(I_n)$.

Proof. Since σ is continuous, $W = \{G\}$ is a linear subspace of $C(I_n)$ (sums and scalar multiples of such sums are again such sums). Suppose W is not dense. By Theorem 8.7 there is $\mu \neq 0$ with $\int h\, d\mu = 0$ for all $h \in W$, in particular

$$\int_{I_n} \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu(\mathbf{x}) = 0 \qquad \text{for all } \mathbf{w} \in \mathbb{R}^n,\ \theta \in \mathbb{R}.$$

Because σ is discriminatory, $\mu = 0$: a contradiction. ■

3.3 Continuous sigmoidals are discriminatory

Theorem 8.9: continuous sigmoidals are discriminatory

Every continuous sigmoidal function is discriminatory for all $\mu \in M(I_n)$.

Proof [Supplement]. Assume $\int \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu = 0$ for all $\mathbf{w}, \theta$. Fix $\mathbf{w}, \theta, \varphi$ and let $\sigma_\lambda(\mathbf{x}) = \sigma(\lambda(\mathbf{w}^\top \mathbf{x} + \theta) + \varphi)$. As $\lambda \to \infty$, $\sigma_\lambda \to 1$ on the open half-space $\Pi = \{\mathbf{w}^\top \mathbf{x} + \theta > 0\}$, $\to 0$ on $\{\mathbf{w}^\top \mathbf{x} + \theta < 0\}$ and $= \sigma(\varphi)$ on the hyperplane $H = \{\mathbf{w}^\top \mathbf{x} + \theta = 0\}$. The σ_λ are bounded by 1, so dominated convergence gives

$$0 = \int \sigma_\lambda\, d\mu \ \longrightarrow\ \mu(\Pi) + \sigma(\varphi)\mu(H) \qquad \text{for every } \varphi.$$

Letting $\varphi \to -\infty$ and $\varphi \to +\infty$ yields $\mu(\Pi) = 0$ and $\mu(H) = 0$ for all half-spaces and hyperplanes. Fix $\mathbf{w}$ and define $F(h) = \int h(\mathbf{w}^\top \mathbf{x})\, d\mu(\mathbf{x})$ for bounded measurable $h : \mathbb{R} \to \mathbb{R}$. F is linear and bounded, and $F(\mathbf{1}_{[\theta,\infty)}) = \mu(\Pi) + \mu(H) = 0$, $F(\mathbf{1}_{(\theta,\infty)}) = 0$, hence F vanishes on indicators of intervals, on simple functions, and by uniform approximation on all bounded measurable h. Taking $h = \cos$ and $h = \sin$ gives $\int e^{i\mathbf{w}^\top \mathbf{x}}\, d\mu(\mathbf{x}) = 0$ for every $\mathbf{w}$: the Fourier transform of μ vanishes, so $\mu = 0$. ■

Proof of Cybenko's theorem. Theorem 8.9 shows σ is discriminatory; Theorem 8.8 gives density. ■

Remark 8.10: what the theorem says and does not say

One hidden layer with enough neurons approximates any continuous function on a compact set uniformly (the cube can be replaced by any compact K after rescaling). The theorem is an existence result: it gives no bound on N, no algorithm to find the weights, and no statement about generalization. Later work extends it to every continuous non-polynomial activation,

including ReLU (Leshno et al. 1993). Intuition in 1D: a steep sigmoid is almost a step, a difference of two steps is a bump, and a sum of bumps approximates any continuous function.

4 Barron's Theorem: a Dimension-Free Rate

4.1 Fourier preliminaries

For $f \in L^1(\mathbb{R}^d)$ the Fourier transform is $\hat{f}(\boldsymbol{\xi}) = \int_{\mathbb{R}^d} f(\mathbf{x}) e^{-2\pi i \boldsymbol{\xi}^\top \mathbf{x}}\, d\mathbf{x}$. The Sobolev space $H^s(\mathbb{R}^d)$ consists of f with $\int |\hat{f}(\boldsymbol{\xi})|^2 (1 + |\boldsymbol{\xi}|^2)^s \, d\boldsymbol{\xi} < \infty$.

Definition 8.11: Barron class

For $C > 0$,

$$\Gamma_C = \Big\{ f \in L^1(\mathbb{R}^d) : \ \|\hat{f}\|_{L^1(\mathbb{R}^d)} < \infty, \ \int_{\mathbb{R}^d} |2\pi\boldsymbol{\xi}| \, |\hat{f}(\boldsymbol{\xi})| \, d\boldsymbol{\xi} < C \Big\}.$$

On a domain Ω one also uses the spectral Barron norm $\|f\|_{\mathcal{B}^s(\Omega)} = \inf_{f_e|_\Omega = f} \int_{\mathbb{R}^d} (1 + |\boldsymbol{\xi}|)^s |\hat{f}_e(\boldsymbol{\xi})| \, d\boldsymbol{\xi}$, the infimum over all extensions f_e of f beyond Ω.

Theorem 8.12 (Theorem 1, Barron 1993)

Let σ be sigmoidal, $f \in \Gamma_C$ and $B_1^d = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| \le 1\}$. Then for every $c > 4C^2$ and every $N \in \mathbb{N}$ there is a network f_N with N neurons such that

$$\frac{1}{|B_1^d|} \int_{B_1^d} |f(\mathbf{x}) - f_N(\mathbf{x})|^2 \, d\mathbf{x} < \frac{c}{N}.$$

The rate $O(1/N)$ does not depend on the dimension d. The price is the assumption $f \in \Gamma_C$, a condition on the Fourier spectrum.

Remark 8.13: curse of dimensionality

Approximating a general function with s bounded derivatives in d variables to accuracy ε by a fixed linear family needs on the order of $\varepsilon^{-d/s}$ terms, which explodes with d. Barron's bound avoids this: the number of neurons needed for mean squared error ε is about c/ε for every d. The dimension enters only through the constant C.

5 Proof of Barron's Theorem

The proof has three steps. Lemma 1 (Maurey) says that an element of a closed convex hull is approximated at rate $1/N$ by averages of N elements. Lemma 2 places every Barron function in such a hull, namely the hull of scaled half-space indicators. A final step replaces the indicators by sigmoids.

5.1 Convex hulls and Maurey's lemma

Definition 8.14: convex hull

For a subset G of a vector space,

$$\operatorname{co}(G) = \Big\{ \sum_{j=1}^{n} \alpha_j x_j : \ n \in \mathbb{N},\ x_j \in G,\ \alpha_j \geq 0,\ \sum_{j=1}^{n} \alpha_j = 1 \Big\},$$

and $\overline{\operatorname{co}}(G)$ denotes its closure in a given norm.

Theorem 8.15 (Lemma 1, Maurey)

Let H be a Hilbert space and $G \subset H$ with $\|g\|_H \leq B$ for all $g \in G$, for some $B > 0$. Let $f \in \overline{\operatorname{co}}(G)$. Then for every $N \in \mathbb{N}$ and every $c > B^2$ there exist $g_1, \dots, g_N \in G$ with

$$\Big\| f - \frac{1}{N} \sum_{i=1}^{N} g_i \Big\|_H^2 \leq \frac{c}{N}.$$

Proof. Fix N and $\varepsilon > 0$. Since $f \in \overline{\operatorname{co}}(G)$ there are $\alpha_1, \dots, \alpha_m \in [0,1]$ summing to 1 and $h_1, \dots, h_m \in G$ such that

$$f^* = \sum_{j=1}^{m} \alpha_j h_j \quad \text{satisfies} \quad \|f - f^*\|_H < \varepsilon.$$

Claim. There are $g_1, \dots, g_N \in \{h_1, \dots, h_m\}$ with $\big\| f^* - \frac{1}{N} \sum_i g_i \big\|_H^2 \leq B^2/N$.

Let $X_1, \dots, X_N$ be iid random elements with $P(X_i = h_j) = \alpha_j$. Then $\mathbb{E}[X_i] = \sum_j P(X_i = h_j)\, h_j = f^*$. Writing $f^* = \frac{1}{N} \sum_{j=1}^{N} f^*$ and using $\|\sum_j v_j\|^2 = \sum_j \|v_j\|^2 + \sum_{i \neq j} \langle v_i, v_j \rangle$,

$$\mathbb{E}\Big\| f^* - \frac{1}{N} \sum_{j=1}^{N} X_j \Big\|^2 = \frac{1}{N^2}\, \mathbb{E}\Big[\sum_{j=1}^{N} \|f^* - X_j\|^2 + \sum_{i \neq j} \langle f^* - X_i, f^* - X_j \rangle \Big].$$

For $i \neq j$, independence gives $\mathbb{E}\langle X_i, X_j \rangle = \langle \mathbb{E}X_i, \mathbb{E}X_j \rangle = \langle f^*, f^* \rangle$, so

$$\mathbb{E}\langle f^* - X_i, f^* - X_j \rangle = \langle f^*, f^* \rangle - \langle f^*, \mathbb{E}X_j \rangle - \langle \mathbb{E}X_i, f^* \rangle + \mathbb{E}\langle X_i, X_j \rangle = 0.$$

The N diagonal terms have the same distribution, hence

$$\mathbb{E}\Big\| f^* - \frac{1}{N} \sum_{j} X_j \Big\|^2 = \frac{1}{N} \mathbb{E}\|f^* - X_1\|^2 = \frac{1}{N} \mathbb{E}[\|f^*\|^2 - 2\langle f^*, X_1 \rangle + \|X_1\|^2]$$

$$= \frac{1}{N} (\mathbb{E}\|X_1\|^2 - \|f^*\|^2) \leq \frac{B^2}{N},$$

using $\mathbb{E}\langle f^*, X_1 \rangle = \|f^*\|^2$ and $\|X_1\| \leq B$. A random variable cannot exceed its mean for every outcome, so some realization $g_i = X_i(\omega) \in \{h_1, \dots, h_m\}$ satisfies the claim.

By the triangle inequality,

$$\Big\| f - \frac{1}{N} \sum_{i} g_i \Big\|_H \leq \|f - f^*\|_H + \Big\| f^* - \frac{1}{N} \sum_{i} g_i \Big\|_H < \varepsilon + \frac{B}{\sqrt{N}}.$$

Since $c > B^2$, choosing $\varepsilon < (\sqrt{c} - B)/\sqrt{N}$ makes the right-hand side less than $\sqrt{c/N}$; squaring gives the lemma. ∎

This is what Theorem 1 needs: if f lies in the closed convex hull of a set of single neurons bounded by B in the normalized L^2 norm, an average of N neurons approximates it with error c/N. So it suffices to show that every $f \in \Gamma_C$ lies in the closed convex hull of affinely transformed Heaviside functions.

5.2 Barron functions lie in a convex hull of half-space indicators

On the unit ball use the normalized norm

$$\|g\|_{L_0^2(B_1^d)} = \Big(\frac{1}{|B_1^d|}\int_{B_1^d} |g(\mathbf{x})|^2\, d\mathbf{x}\Big)^{1/2},$$

and the set of scaled half-space indicators

$$G_C = \Big\{\mathbf{x}\in B_1^d \mapsto \gamma\, \mathbf{1}_{\mathbb{R}_+}(\langle \mathbf{a},\mathbf{x}\rangle + b) :\ \mathbf{a}\in\mathbb{R}^d,\ b\in\mathbb{R},\ |\gamma|\le 2C\Big\},$$

where $\mathbf{1}_{\mathbb{R}_+}(t) = 1$ for $t \ge 0$ and 0 otherwise (the Heaviside function). Every $g \in G_C$ has $\|g\|_{L_0^2(B_1^d)} \le |\gamma| \le 2C$.

Theorem 8.16 (Lemma 2, Barron 1993)

Let $d \in \mathbb{N}$, $C > 0$ and $f \in \Gamma_C$. Then

$$f|_{B_1^d} - f(\mathbf{0}) \in \overline{\mathrm{co}}(G_C),$$

where the closure is taken in $L_0^2(B_1^d)$.

Proof. Write $K = \overline{\mathrm{co}}(G_C)$ (closure in $L_0^2(B_1^d)$) and $h = f|_{B_1^d} - f(\mathbf{0})$. As a closed convex hull, K is a closed convex subset of the Hilbert space $L_0^2(B_1^d)$. The lecture gave a sketch: the integral representation of Step 1 and the idea that a Riemann sum of it is a convex combination. Parts marked [Supplement] are not in the lecture notes; they fill the gaps of the sketch, following the structure of Barron's original proof.

Step 1 (integral representation). Since $f \in \Gamma_C$, both f and $\hat f$ are in $L^1(\mathbb{R}^d)$, so Fourier inversion holds:

$$f(\mathbf{x}) - f(\mathbf{0}) = \int_{\mathbb{R}^d} \hat f(\boldsymbol{\xi})(e^{2\pi i\langle \mathbf{x},\boldsymbol{\xi}\rangle} - 1)\, d\boldsymbol{\xi}.$$

Write $\hat f(\boldsymbol{\xi}) = |\hat f(\boldsymbol{\xi})| e^{i\kappa(\boldsymbol{\xi})}$, where $\kappa(\boldsymbol{\xi})$ is the phase. Then

$$f(\mathbf{x}) - f(\mathbf{0}) = \int_{\mathbb{R}^d} |\hat f(\boldsymbol{\xi})|(e^{i(2\pi\langle \mathbf{x},\boldsymbol{\xi}\rangle + \kappa(\boldsymbol{\xi}))} - e^{i\kappa(\boldsymbol{\xi})})\, d\boldsymbol{\xi}.$$

For real-valued f the left side is real, so we may take real parts:

$$f(\mathbf{x}) - f(\mathbf{0}) = \int_{\mathbb{R}^d} \frac{\cos\big(2\pi\langle \mathbf{x},\boldsymbol{\xi}\rangle + \kappa(\boldsymbol{\xi})\big) - \cos\kappa(\boldsymbol{\xi})}{|2\pi\boldsymbol{\xi}|}\, |2\pi\boldsymbol{\xi}|\, |\hat f(\boldsymbol{\xi})|\, d\boldsymbol{\xi}.$$

At $\boldsymbol{\xi} = \mathbf{0}$ the original integrand vanishes, so the division is harmless. Let $C_f = \int |2\pi\boldsymbol{\xi}||\hat f(\boldsymbol{\xi})|\, d\boldsymbol{\xi} < C$ and define the probability measure Λ on $\mathbb{R}^d$ by $d\Lambda(\boldsymbol{\xi}) = C_f^{-1}|2\pi\boldsymbol{\xi}||\hat f(\boldsymbol{\xi})|\, d\boldsymbol{\xi}$ (the lecture writes C for C_f; if $C_f = 0$ then $\hat f = 0$ a.e. off the origin, $h = 0 \in K$ and there is nothing to prove). With

$$q_{\boldsymbol{\xi}}(\mathbf{x}) = \frac{\cos\big(2\pi\langle \mathbf{x},\boldsymbol{\xi}\rangle + \kappa(\boldsymbol{\xi})\big) - \cos\kappa(\boldsymbol{\xi})}{|2\pi\boldsymbol{\xi}|}, \qquad k_{\boldsymbol{\xi}} = C_f q_{\boldsymbol{\xi}},$$

we obtain

$$f(\mathbf{x}) - f(\mathbf{0}) = C_f \int_{\mathbb{R}^d} q_{\boldsymbol{\xi}}(\mathbf{x})\, d\Lambda(\boldsymbol{\xi}) = \int_{\mathbb{R}^d} k_{\boldsymbol{\xi}}(\mathbf{x})\, d\Lambda(\boldsymbol{\xi}). \tag{5}$$

Step 2 (each $k_{\boldsymbol{\xi}}$ lies in K). [Supplement]

For $\boldsymbol{\xi} = \mathbf{0}$ put $k_{\mathbf{0}} = 0$, which is the element of G_C with $\gamma = 0$. Fix $\boldsymbol{\xi} \ne \mathbf{0}$ and set $\mathbf{u} = \boldsymbol{\xi}/|\boldsymbol{\xi}|$ and $t = \langle \mathbf{u},\mathbf{x}\rangle$. By Cauchy–Schwarz, $t \in [-1,1]$ on the ball. Then $k_{\boldsymbol{\xi}}(\mathbf{x}) = r(t)$ with

$$r(t) = C_f \frac{\cos\big(2\pi|\boldsymbol{\xi}|t + \kappa\big) - \cos\kappa}{2\pi|\boldsymbol{\xi}|}, \qquad r(0) = 0, \qquad |r'(t)| = C_f|\sin\big(2\pi|\boldsymbol{\xi}|t + \kappa\big)| \le C_f,$$

so $|r(s) - r(t)| \le C_f|s-t|$ by the mean value theorem.

Staircase. For $m \in \mathbb{N}$ let $t_k = k/m$ $(k = 0, \dots, m)$ and

$$s_m(t) = \sum_{k=1}^{m} \alpha_k \mathbf{1}_{\mathbb{R}_+}(t - t_k) + \sum_{k=1}^{m} \alpha'_k \mathbf{1}_{\mathbb{R}_+}(-t - t_k), \qquad \alpha_k = r(t_k) - r(t_{k-1}), \;\; \alpha'_k = r(-t_k) - r(-t_{k-1}).$$

If $t \in [t_j, t_{j+1})$ with $t \ge 0$, every term of the second sum is 0 (its threshold t_k is positive), and the first sum telescopes to $r(t_j) - r(0) = r(t_j)$. Symmetrically, for $t < 0$ the first sum vanishes and the second gives $r(-t_j)$ for $-t \in [t_j, t_{j+1})$. In both cases t is within $1/m$ of the grid point used, so

$$\sup_{t \in [-1,1]} |r(t) - s_m(t)| \le \frac{C_f}{m}.$$

Coefficients. Each $|\alpha_k|, |\alpha'_k| \le C_f/m$, so $S := \sum_k (|\alpha_k| + |\alpha'_k|) \le 2C_f \le 2C$.

Convex combination. In terms of $\mathbf{x}$, $\mathbf{1}_{\mathbb{R}_+}(t - t_k) = \mathbf{1}_{\mathbb{R}_+}(\langle \mathbf{u}, \mathbf{x}\rangle - t_k)$ and $\mathbf{1}_{\mathbb{R}_+}(-t - t_k) = \mathbf{1}_{\mathbb{R}_+}(\langle -\mathbf{u}, \mathbf{x}\rangle - t_k)$ are half-space indicators. List the $2m$ coefficients as β_j and the matching indicators as e_j. If $S = 0$ then $s_m = 0 \in G_C$. Otherwise

$$s_m(\langle \mathbf{u}, \cdot \rangle) = \sum_j \frac{|\beta_j|}{S} \Big(S \, \operatorname{sign}(\beta_j) \, e_j \Big),$$

with weights $|\beta_j|/S \ge 0$ summing to 1 and each bracket in G_C because $S \le 2C$. So $s_m(\langle \mathbf{u}, \cdot \rangle) \in \operatorname{co}(G_C)$.

Limit. The normalized norm is bounded by the sup norm, $\|g\|_{L_0^2(B_1^d)} \le \sup_{B_1^d} |g|$, because the normalized norm averages $|g|^2$ over the ball against the uniform probability measure. Hence $\|k_{\boldsymbol{\xi}} - s_m(\langle \mathbf{u}, \cdot \rangle)\|_{L_0^2} \le C_f/m \to 0$, and $k_{\boldsymbol{\xi}} \in K$.

Step 3 (the average stays in K). The lecture expresses this step by a Riemann sum: partition $\mathbb{R}^d$ into cubes $I_{\boldsymbol{\theta}}$ ($\boldsymbol{\theta} \in \frac{1}{n}\mathbb{Z}^d$); then $\sum_{\boldsymbol{\theta}} k_{\boldsymbol{\theta}} \Lambda(I_{\boldsymbol{\theta}})$ is a convex combination of elements of K approximating the integral (5). Making this rigorous needs a truncation of the infinite sum and continuity of $\boldsymbol{\xi} \mapsto k_{\boldsymbol{\xi}}$, which can fail because the phase κ may jump where $\hat{f} = 0$. The following argument avoids both issues. [Supplement]

Projection. K is nonempty, closed and convex in a Hilbert space, so h has a nearest point $p \in K$, characterized by

$$\langle h - p,\, k - p \rangle \le 0 \qquad \text{for all } k \in K.$$

Averaging. Since Λ is a probability measure, (5) gives $h(\mathbf{x}) - p(\mathbf{x}) = \int \big(k_{\boldsymbol{\xi}}(\mathbf{x}) - p(\mathbf{x})\big)\, d\Lambda(\boldsymbol{\xi})$. Take the inner product with $h - p$ and exchange the integrals over B_1^d and over Λ by Fubini; this is allowed because $|k_{\boldsymbol{\xi}}| \le C_f$ (Step 2, since $|r(t)| \le C_f|t| \le C_f$) and L^2 functions on the ball are integrable. With Step 2,

$$\|h - p\|_{L_0^2}^2 = \int_{\mathbb{R}^d} \langle h - p,\, k_{\boldsymbol{\xi}} - p \rangle \, d\Lambda(\boldsymbol{\xi}) \le 0.$$

Hence $h = p \in K$. ∎

5.3 Completing the proof of Theorem 1

Proof of Barron's theorem. Let $f \in \Gamma_C$. By Lemma 8.16, $f|_{B_1^d} - f(\mathbf{0}) \in \overline{\operatorname{co}}(G_C)$, and every $g \in G_C$ has $\|g\|_{L_0^2(B_1^d)} \le 2C$. Lemma 8.15 with $H = L_0^2(B_1^d)$ and $B = 2C$ gives, for every $N \in \mathbb{N}$ and $c > 4C^2$, parameters $|\gamma_i| \le 2C$, $\mathbf{a}_i$, b_i with

$$\frac{1}{|B_1^d|} \int_{B_1^d} \Big| f(\mathbf{x}) - f(\mathbf{0}) - \frac{1}{N} \sum_{i=1}^{N} \gamma_i \mathbf{1}_{\mathbb{R}_+}\big(\langle \mathbf{a}_i, \mathbf{x}\rangle + b_i\big) \Big|^2 d\mathbf{x} < \frac{c}{N}.$$

Finally a sigmoid becomes a step under scaling: $\sigma(\lambda t) \to \mathbf{1}_{\mathbb{R}_+}(t)$ as $\lambda \to \infty$ for $t \neq 0$, and the hyperplanes $\{\langle \mathbf{a}_i, \mathbf{x}\rangle + b_i = 0\}$ have measure zero. (A hyperplane is a null set only when $\mathbf{a}_i \neq \mathbf{0}$. If $\mathbf{a}_i = \mathbf{0}$ and $b_i = 0$ the set is all of $\mathbb{R}^d$; that term is the constant γ_i, which is also obtained with $\mathbf{a}_i = \mathbf{0}$, $b_i = 1$, so we may assume this case does not occur.) [Supplement]By dominated convergence, replacing each indicator by $\sigma\big(\lambda(\langle \mathbf{a}_i, \mathbf{x}\rangle + b_i)\big)$ with λ large keeps the error below c/N. The result

$$f_N(\mathbf{x}) = f(\mathbf{0}) + \sum_{i=1}^{N} \frac{\gamma_i}{N}\, \sigma\big(\lambda\langle \mathbf{a}_i, \mathbf{x}\rangle + \lambda b_i\big)$$

is a one-hidden-layer network with N neurons; the constant $f(\mathbf{0})$ is the output bias. ■

6 Generalization and Rademacher Complexity

6.1 Population and empirical risk

Let $\mathbf{x} \in \mathbb{R}^d$ be the input, $y \in \mathcal{Y}$ the output, and $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ iid training data from a distribution P. For a loss ℓ and a model class $\mathcal{F}$ we want

$$\arg\min_{f \in \mathcal{F}} \ \mathbb{E}_{(\mathbf{x},y)\sim P}\big[\ell(y, f(\mathbf{x}))\big] \quad \text{(population risk)},$$

but can only compute

$$\arg\min_{f \in \mathcal{F}} \ \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f(\mathbf{x}_i)) \quad \text{(empirical risk)}.$$

The generalization error of Section 1 is controlled by how far empirical averages can be from expectations uniformly over the class, $\sup_{f\in\mathcal{F}} \big|\frac{1}{n}\sum_i f(\mathbf{z}_i) - \mathbb{E} f\big|$, applied to the class of losses $\mathbf{z} = (\mathbf{x}, y) \mapsto \ell(y, f(\mathbf{x}))$.

6.2 Rademacher complexity

Definition 8.17: Rademacher variables and complexity

Random variables $\xi_1, \dots, \xi_n$ are **Rademacher** if they are iid with $P(\xi_i = 1) = P(\xi_i = -1) = \frac{1}{2}$. For a class $\mathcal{F}$ and iid samples $\mathbf{z}_1, \dots, \mathbf{z}_n$, the **Rademacher complexity** is

$$\mathrm{Rad}_n(\mathcal{F}) = \mathbb{E}\Big[\sup_{f\in\mathcal{F}} \frac{1}{n}\sum_{i=1}^{n} \xi_i f(\mathbf{z}_i)\Big],$$

the expectation being over both the ξ_i and the $\mathbf{z}_i$.

It measures how well the class can correlate with random signs: a rich class fits noise and has large complexity.

Theorem 8.18: uniform deviation bound

By symmetrization, $\mathbb{E}\sup_{f\in\mathcal{F}} \big|\frac{1}{n}\sum_i f(\mathbf{z}_i) - \mathbb{E} f\big| \le 2\,\mathrm{Rad}_n(\mathcal{F} \cup -\mathcal{F})$. If moreover every $f \in \mathcal{F}$ takes values in $[0, 1]$, then with probability at least $1 - \delta$

$$\sup_{f\in\mathcal{F}} \Big|\frac{1}{n}\sum_{i=1}^{n} f(\mathbf{z}_i) - \mathbb{E} f\Big| \le 2\,\mathrm{Rad}_n(\mathcal{F} \cup -\mathcal{F}) + \sqrt{\frac{\log(2/\delta)}{2n}}.$$

The lecture records this as "$\le 2\,\mathrm{Rad}(\mathcal{F}) + 1/\sqrt{2n}$", suppressing the confidence factor.

6.3 Two-layer networks

For $\mathcal{F} = \{f_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_j \beta_j \sigma(\mathbf{w}_j^\top \mathbf{x})\}$ with $\sum_j |\beta_j| \le Q$, $\|\mathbf{w}_j\| \le W$, inputs $\|\mathbf{x}\| \le R$ and σ 1-Lipschitz with $\sigma(0) = 0$ (e.g. tanh), the contraction lemma gives

$$\mathrm{Rad}_n(\mathcal{F}) \le \frac{2QWR}{\sqrt{n}} = O\Big(\frac{1}{\sqrt{n}}\Big).$$

The bound depends on the size Q of the outer weights, not on the number of neurons. Together with Barron's theorem, the approximation error decays like $1/N$ and the generalization error like $1/\sqrt{n}$, and neither rate depends on d.

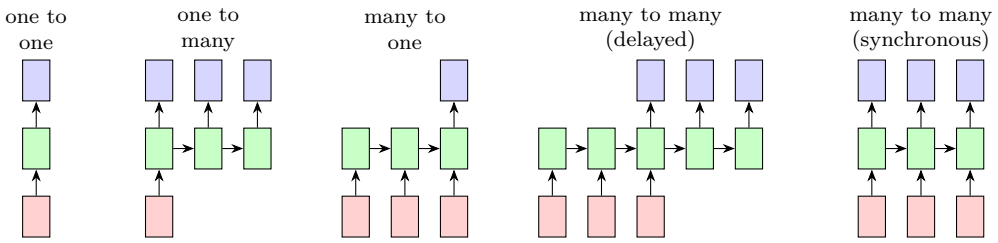
Chapter 9. Recurrent Neural Networks and LSTM

Lecture 10/28: RNNs, backpropagation through time, vanishing and exploding gradients, LSTM, language modeling, image captioning

1 Why Recurrent Networks

A feedforward network maps one fixed-size input to one fixed-size output. Many tasks instead consume or produce **sequences**, often of variable length. A recurrent network handles them by reading one element at a time and carrying an **internal (hidden) state** that is updated as the sequence is processed. The input/output patterns seen in lecture:

Pattern	Input → output	Examples
one to one	single → single	vanilla NN / FFN
one to many	single → sequence	image to video, image captioning
many to one	sequence → single	video classification, action recognition
many to many (delayed)	read all, then emit	translation (English to Korean), video to caption
many to many (synchronous)	emit at every step	video segmentation, video editing, style transfer



Inputs (red), hidden states (green) and outputs (blue); horizontal arrows carry the state.

In the delayed pattern the outputs start only after the whole input has been read (e.g. inputs at $t = 1, 2, 3$, outputs at $t = 4, 5, 6$).

2 The Recurrence

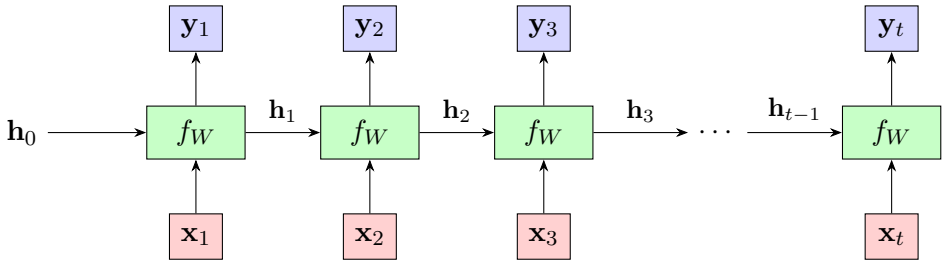
Definition 9.1: recurrent neural network

Given inputs $\mathbf{x}_1, \mathbf{x}_2, \ldots$, an RNN computes hidden states and outputs by

$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t), \qquad \mathbf{y}_t = f_{W_{hy}}(\mathbf{h}_t), \qquad t \geq 1,$$

starting from an initial state $\mathbf{h}_0$ (usually $\mathbf{h}_0 = \mathbf{0}$). The functions f_W (new state from old state and current input) and $f_{W_{hy}}$ (output from state) and their parameters are **the same at every time step**.

Unrolling the loop over time gives an ordinary computational graph with one copy of the cell per time step, all copies sharing parameters. Weight sharing is what lets one network process sequences of any length.



The unrolled RNN: every copy of the cell uses the same f_W, and the output map $f_{W_{hy}}$ is shared as well.

Definition 9.2: vanilla RNN

With an elementwise activation σ (typically tanh),

$$\mathbf{h}_t = \sigma(U\mathbf{h}_{t-1} + W\mathbf{x}_t + \mathbf{b}), \qquad \hat{\mathbf{y}}_t = W^{\text{out}}\mathbf{h}_t, \qquad \mathbf{h}_0 = \mathbf{0}.$$

Written out for the first two steps:

$$\mathbf{h}_1 = \sigma(W\mathbf{x}_1 + \mathbf{b}), \quad \hat{\mathbf{y}}_1 = W^{\text{out}}\mathbf{h}_1; \qquad \mathbf{h}_2 = \sigma(U\mathbf{h}_1 + W\mathbf{x}_2 + \mathbf{b}), \quad \hat{\mathbf{y}}_2 = W^{\text{out}}\mathbf{h}_2.$$

The same U, W, $\mathbf{b}$, W^{out} appear at both steps. The letters for the two weight matrices are not standardized: the next section (following the handwritten lecture notes) writes W for the recurrent weight and U for the input weight.

3 Backpropagation Through Time

3.1 The idea

The loss of a sequence is a sum of per-step losses, $L = \sum_{t=1}^{\tau} L_t$. Each L_t depends on the parameters through the current step and, via the hidden state, through every earlier step. **Backpropagation through time (BPTT)** is ordinary backpropagation on the unrolled graph. One forward pass runs through the entire sequence to compute L; one backward pass runs through the entire sequence to compute the gradient. Because parameters are shared, the gradient with respect to a parameter is the sum of the contributions from all of its copies.

Remark 9.3: truncated BPTT

For long sequences the full backward pass is expensive. **Truncated BPTT** carries the hidden state forward through the whole sequence but backpropagates only through a window of the most recent steps. Gradients from beyond the window are ignored.

3.2 A worked two-step example

Consider the scalar RNN

$$h_t = \tanh(Wh_{t-1} + Ux_t + b), \qquad Y_t = Vh_t + c, \qquad \theta = (W, U, V, b, c),$$

with inputs $(x_1, \ldots, x_\tau)$, outputs $(Y_1, \ldots, Y_\tau)$, targets $(Z_1, \ldots, Z_\tau)$ and loss $L = \sum_t L_t$ with $L_t = \frac{1}{2}(Y_t - Z_t)^2$. To minimize L we need $\nabla_\theta L$. The chain rule now acts both within each step and across time, which is what makes this harder than for an MLP. Take $\tau = 2$ and all quantities one-dimensional. The forward pass is

$$\begin{aligned} a_1 &= Wh_0 + Ux_1 + b, \quad h_1 = \tanh a_1, \quad Y_1 = Vh_1 + c, \\ a_2 &= Wh_1 + Ux_2 + b, \quad h_2 = \tanh a_2, \quad Y_2 = Vh_2 + c, \end{aligned} \qquad L = \tfrac{1}{2}(Y_1 - Z_1)^2 + \tfrac{1}{2}(Y_2 - Z_2)^2.$$

Local derivatives. Using $\tanh'(a) = \operatorname{sech}^2 a = 1 - \tanh^2 a$, and writing ∂ for the derivative with respect to the explicit occurrence (other arguments held fixed):

$$\frac{\partial h_t}{\partial W} = (1 - h_t^2)h_{t-1}, \qquad \frac{\partial h_t}{\partial U} = (1 - h_t^2)x_t, \qquad \frac{\partial h_t}{\partial b} = 1 - h_t^2, \qquad \frac{\partial h_2}{\partial h_1} = (1 - h_2^2)W.$$

Since L_t depends on h_t only through Y_t,

$$\frac{\partial L_t}{\partial h_t} = \frac{\partial L_t}{\partial Y_t}\frac{\partial Y_t}{\partial h_t} = (Y_t - Z_t)V.$$

Output parameters. L_t depends on V and c only through Y_t, so

$$\frac{\partial L}{\partial V} = \sum_{t=1}^{2}(Y_t - Z_t)h_t, \qquad \frac{\partial L}{\partial c} = \sum_{t=1}^{2}(Y_t - Z_t).$$

Recurrent weight. L_1 sees W through h_1. L_2 sees W directly through h_2 and again through h_1, which feeds h_2:

$$\frac{\partial L}{\partial W} = \frac{\partial L_1}{\partial h_1}\frac{\partial h_1}{\partial W} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial W} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W},$$

that is,

$$\frac{\partial L}{\partial W} = (Y_1 - Z_1)V(1 - h_1^2)h_0 + (Y_2 - Z_2)V(1 - h_2^2)h_1 + (Y_2 - Z_2)V(1 - h_2^2)W(1 - h_1^2)h_0.$$

Input weight and bias [Supplement]. They have the same three-path structure, with h_{t-1} replaced by x_t and by 1 respectively:

$$\frac{\partial L}{\partial U} = (Y_1 - Z_1)V(1 - h_1^2)x_1 + (Y_2 - Z_2)V(1 - h_2^2)x_2 + (Y_2 - Z_2)V(1 - h_2^2)W(1 - h_1^2)x_1,$$
$$\frac{\partial L}{\partial b} = (Y_1 - Z_1)V(1 - h_1^2) + (Y_2 - Z_2)V(1 - h_2^2) + (Y_2 - Z_2)V(1 - h_2^2)W(1 - h_1^2).$$

With $h_0 = 0$ the first and third terms of $\partial L/\partial W$ vanish, but the other two gradients keep all three terms.

3.3 General formulation

Let the input be a sequence $\tau(X) = (u_1, \ldots, u_L)$ with $u_\ell \in \mathbb{R}^m$. The state update is a map $q_{\tilde\theta} : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$, e.g. $q_{\tilde\theta}(h, u) = \phi(Wh + Uu + c)$ with $\tilde\theta = (W, U, c) \in \mathbb{R}^p$:

$$h_0 = \mathbf{0}, \qquad h_\ell = q_{\tilde\theta}(h_{\ell-1}, u_\ell) \quad (\ell = 1, \ldots, L), \qquad f_\theta(X) = Ah_L + b,$$

with $A \in \mathbb{R}^{1\times n}$, $b \in \mathbb{R}$, $h_\ell \in \mathbb{R}^n$ and $\theta = (A, b, \tilde\theta)$. For a loss $\mathcal{L}(f_\theta(X), Y)$,

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial f}(f_\theta(X), Y)\,\frac{\partial f_\theta(X)}{\partial \theta}, \qquad \frac{\partial f_\theta(X)}{\partial A} = h_L^\top, \quad \frac{\partial f_\theta(X)}{\partial b} = 1, \quad \frac{\partial f_\theta(X)}{\partial \tilde\theta} = A\frac{dh_L}{d\tilde\theta}.$$

Here $dh_L/d\tilde\theta$ is the total derivative, while $\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell)$ below holds $h_{\ell-1}$ fixed. Write $J_k = \partial_h q_{\tilde\theta}(h_{k-1}, u_k) \in \mathbb{R}^{n\times n}$ for the state-to-state Jacobian.

Theorem 9.4: BPTT formula

For $\ell \le L$,

$$\frac{\partial h_L}{\partial h_\ell} = J_L J_{L-1} \cdots J_{\ell+1}, \qquad \frac{dh_L}{d\tilde\theta} = \sum_{\ell=1}^{L}\frac{\partial h_L}{\partial h_\ell}\,\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell),$$

(the empty product for $\ell = L$ is the identity). Consequently

$$\frac{\partial \mathcal{L}(f_\theta(X), Y)}{\partial \tilde{\theta}} = \sum_{\ell=1}^{L} \frac{\partial \mathcal{L}(f_\theta(X), Y)}{\partial h_\ell} \, \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{\ell-1}, u_\ell), \qquad \frac{\partial \mathcal{L}}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} \frac{\partial h_L}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} J_L \cdots J_{\ell+1}.$$

Proof. Step 1 (state Jacobian). For $k > \ell$, h_k depends on h_ℓ only through h_{k-1}, so $\partial h_k / \partial h_\ell = J_k \, \partial h_{k-1} / \partial h_\ell$. Starting from $\partial h_\ell / \partial h_\ell = I$ and iterating gives the product.

Step 2 (one-step recursion). h_ℓ depends on $\tilde{\theta}$ directly and through $h_{\ell-1}$, so the chain rule gives

$$\frac{dh_\ell}{d\tilde{\theta}} = \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{\ell-1}, u_\ell) + J_\ell \frac{dh_{\ell-1}}{d\tilde{\theta}}, \qquad \frac{dh_0}{d\tilde{\theta}} = 0.$$

Step 3 (unrolling). Induction on L. For $L = 1$ the formula reads $dh_1/d\tilde{\theta} = \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_0, u_1)$, which is Step 2. If the formula holds for $L - 1$, Step 2 and Step 1 give

$$\frac{dh_L}{d\tilde{\theta}} = \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{L-1}, u_L) + \sum_{\ell=1}^{L-1} J_L \frac{\partial h_{L-1}}{\partial h_\ell} \, \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{\ell-1}, u_\ell) = \sum_{\ell=1}^{L} \frac{\partial h_L}{\partial h_\ell} \, \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{\ell-1}, u_\ell).$$

Step 4 (loss). $\mathcal{L}$ depends on $\tilde{\theta}$ only through h_L. Multiplying by $\partial \mathcal{L}/\partial h_L$ and using $\partial \mathcal{L}/\partial h_\ell = (\partial \mathcal{L}/\partial h_L)(\partial h_L/\partial h_\ell)$ gives the last two identities. ■

The sum over ℓ is the parameter-sharing sum: each time step contributes the local derivative of its own copy of q, weighted by how strongly h_ℓ influences the loss.

4 Vanishing and Exploding Gradients

4.1 RNNs are very deep networks

Sequence lengths of hundreds or thousands are common, so the factor $\partial h_L / \partial h_\ell = J_L \cdots J_{\ell+1}$ in Theorem 9.4 is a product of up to $L - \ell$ Jacobians. Multiplying many numbers is unstable. If most factors exceed 1 the product blows up (**exploding gradients**). If most are below 1 it tends to 0 (**vanishing gradients**, the bigger problem). Only factors close to 1 give a product of reasonable size. For the scalar tanh RNN of Section 3.2 each factor is $\partial h_t / \partial h_{t-1} = (1 - h_t^2) W$.

Theorem 9.5: a sufficient condition for vanishing gradients

Let $q_{\tilde{\theta}}(h, u) = \tanh(Wh + Uu + c)$ and let $\|\cdot\|$ be the spectral norm. Then

$$\left\| \frac{\partial h_L}{\partial h_\ell} \right\| \le \|W\|^{L-\ell}.$$

In particular, if $\|W\| < 1$ the gradient signal from step L to step ℓ decays geometrically in $L - \ell$.

Proof. Step 1. With $a_k = W h_{k-1} + U u_k + c$, the Jacobian is $J_k = \operatorname{diag}(1 - \tanh^2(a_k)) \, W = \operatorname{diag}(1 - h_k^2) \, W$.

Step 2. The diagonal entries lie in $[0, 1]$, so $\|\operatorname{diag}(1 - h_k^2)\| \le 1$ and $\|J_k\| \le \|W\|$.

Step 3. Submultiplicativity of the spectral norm and Theorem 9.4 give $\|J_L \cdots J_{\ell+1}\| \le \prod_{k=\ell+1}^{L} \|J_k\| \le \|W\|^{L-\ell}$. ■

The converse fails: $\|W\| > 1$ does not force explosion, because saturated units ($h_k^2 \approx 1$) shrink the factors. Exploding gradients need $\|W\|$ large and units away from saturation.

4.2 Exploding gradients and gradient clipping

A very large gradient means the output is very sensitive to small parameter changes in some direction. Sometimes such gradients are unworkable and the architecture must be changed. Sometimes, however, the direction is fine and only the length is the problem. Then one uses **gradient clipping** with a scalar threshold $v > 0$:

$$g \leftarrow \min\Big(1, \frac{v}{\|g\|}\Big) g = \begin{cases} g, & \|g\| \le v, \\ \dfrac{v}{\|g\|}\, g, & \text{otherwise.} \end{cases}$$

The clipped vector has the same direction as g and length $\min(\|g\|, v)$. The slide writes max in the first expression; the case split shows that min is meant. On a loss surface with a steep cliff, an unclipped step can jump far away from the valley, while a clipped step stays close.

4.3 Vanishing gradients

In an RNN $\partial \mathcal{L}/\partial h_L$ may be of reasonable size while

$$\frac{\partial \mathcal{L}}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} \frac{\partial h_L}{\partial h_{L-1}} \frac{\partial h_{L-1}}{\partial h_{L-2}} \cdots \frac{\partial h_{\ell+1}}{\partial h_\ell}$$

is tiny. Then small changes of h_ℓ barely affect $\mathcal{L}$. Since

$$\frac{\partial \mathcal{L}}{\partial u_\ell} = \frac{\partial \mathcal{L}}{\partial h_\ell} \frac{\partial h_\ell}{\partial u_\ell},$$

small changes of the input u_ℓ barely affect $\mathcal{L}$ either: loosely, the network does not remember u_ℓ at step L. (This is not exact, since large changes of u_ℓ may still matter.) In any case the gradient signal from time L is lost by time ℓ, so the model cannot learn which information to keep at time ℓ.

5 Long Short-Term Memory

Motivation. Plain RNNs cannot learn long-range dependencies because gradients vanish over many time steps. The LSTM adds a memory cell updated additively through gates, so gradients can flow across long time spans.

5.1 The LSTM cell

The vanilla RNN computes $h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$. The LSTM keeps a second state, the **cell state** c_t (long memory), next to the hidden state h_t, and controls it with gates.

Definition 9.6: LSTM

With σ the logistic sigmoid and $\odot$ elementwise multiplication,

$$\begin{aligned} i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), & f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), & g_t &= \tanh(W_g x_t + U_g h_{t-1} + b_g), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

The four gates have the following roles.

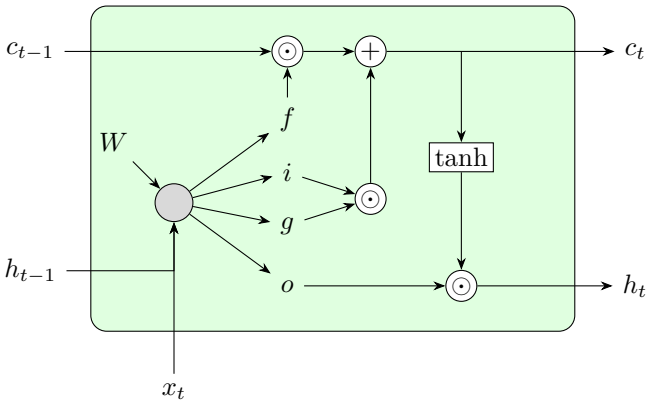
- Input gate i_t: how much of the new content g_t to write into the cell.

- Forget gate f_t: how much of the old cell c_{t-1} to carry over.
- Output gate o_t: how much of the cell to reveal in h_t.
- Candidate ("gate gate") g_t: the new content computed from the current input.

Stacking the four pre-activations gives the compact form

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix},$$

with one matrix W of size $4h \times 2h$ when the hidden state and the input both have dimension h (in general $4h \times (h+m)$ for input dimension m). Its output has length $4h$ and is split into four blocks of length h. Another common notation concatenates $[h_{t-1}, x_t]$, writes $\tilde{C}_t$ for g_t and $*$ for $\odot$.



The LSTM cell: W acts on the stacked (h_{t-1}, x_t); the cell state runs along the top.

5.2 Gradient flow through the cell

Theorem 9.7: the cell path

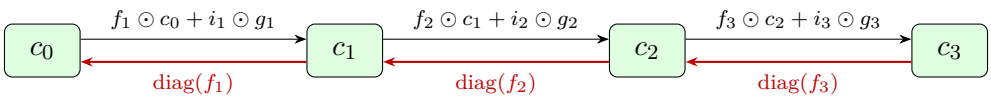
Holding the gates fixed, $\dfrac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(f_t)$. Along the path that goes only through cell states,

$$\frac{\partial c_L}{\partial c_\ell} = \operatorname{diag}\Big(\prod_{k=\ell+1}^{L} f_k\Big) \qquad \text{(product taken entrywise)}.$$

Proof. Step 1. In $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ the second term does not contain c_{t-1}, and the first is the diagonal linear map $c_{t-1} \mapsto \operatorname{diag}(f_t) c_{t-1}$.

Step 2. Chaining these Jacobians and multiplying diagonal matrices entrywise gives the product formula. ■

Compare the vanilla RNN, where each factor is $\operatorname{diag}(1 - h_t^2)W$: the same matrix W is multiplied in at every step and the tanh derivative can only shrink it. In the LSTM the factor is the forget gate itself, chosen by the network at each step, with no repeated matrix product and no squashing derivative. If $f_t \approx 1$ the gradient passes through many steps almost unchanged ("uninterrupted gradient flow"). For example, with $f = 1$ and $i = 0$ the content of the cell is preserved indefinitely. For a vanilla RNN it is much harder to learn a recurrent matrix W that preserves information in the hidden state.



Forward (black) and backward (red) along the cell states: the backward signal is only multiplied by forget gates.

Remark 9.8: what the LSTM does not guarantee

The full Jacobian $\partial c_L / \partial c_\ell$ also has paths through h_{t-1} and the gates, and f_t may be far from 1. So the LSTM does not rule out vanishing or exploding gradients. It gives the model an easier way to learn long-range dependencies.

6 Applications

The lecture covered two applications: language modeling with RNNs, and image captioning. As a further example it mentioned SIRD epidemic models. There the dynamics are an ODE whose infection rate α changes over time, and an LSTM can learn the sequence of α from data.

6.1 Language modeling

A sentence is a sequence of discrete symbols (tokens). Sentence generation, machine translation and text summarization are all instances of modeling language with a network f_θ.

Definition 9.9: one-hot encoding (tokenization)

With a vocabulary of n tokens, token t is represented by $\mathbf{x}_t \in \mathbb{R}^n$ having a single 1 at the index of that token and zeros elsewhere.

Definition 9.10: RNN language model

Sentence generation is next-token prediction. At step t:

$$\begin{aligned} \mathbf{z}_t &= W_e \mathbf{x}_t + \mathbf{b}_e, & W_e &\in \mathbb{R}^{d \times n}, \\ \mathbf{h}_t &= \mathrm{LSTM}(\mathbf{z}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}; \theta), & \mathbf{h}_t &\in \mathbb{R}^d, \\ \hat{\mathbf{x}}_{t+1} &= W_d \mathbf{h}_t + \mathbf{b}_d, & W_d &\in \mathbb{R}^{n \times d}. \end{aligned}$$

The three lines are the continuous word embedding, the recurrent network (whose state is the continuous output embedding), and the classification layer giving scores over the vocabulary. Generation starts from the start-of-sentence token `<SOS>` as $\mathbf{x}_0$ and ends at the end-of-sentence token `<EOS>`.

Training. The model reads the true sentence shifted by one, `<SOS>`, $\mathbf{x}_1, \ldots, \mathbf{x}_{T-1}$, and at each step predicts the next token. The loss is a word-level classification loss,

$$\sum_{t=1}^{T} \mathcal{L}(\hat{\mathbf{x}}_t, \mathbf{x}_t), \qquad \mathcal{L}(\hat{\mathbf{x}}_t, \mathbf{x}_t) = -\log \mathrm{softmax}(\hat{\mathbf{x}}_t)_{k_t},$$

where k_t is the index of the true token. The model's sentence probability factors as

$$p_\theta(\mathbf{x}_1, \ldots, \mathbf{x}_T) = \prod_{t=1}^{T} p_\theta(\mathbf{x}_t \mid \mathbf{x}_0, \ldots, \mathbf{x}_{t-1}).$$

So minimizing the summed cross-entropy is maximum likelihood for this probability (Chapter 2).

Inference. At each step take the output scores $S(\mathbf{x}_t)$ and choose a token by one of two rules.

- Greedy (deterministic): $\mathbf{x}_t = \arg\max S(\mathbf{x}_t)$.
- Probabilistic (stochastic): sample $\mathbf{x}_t \sim S(\mathbf{x}_t)$ from the softmax probabilities.

Feed the chosen token back as the next input, and stop when `<EOS>` is sampled. Greedy decoding always returns the same sentence. Sampling can also pick less likely words, so repeated runs give different sentences.

Machine translation reads a sentence in one language and generates its translation in another. This is the delayed many-to-many pattern.

6.2 Image captioning

The task is to describe an image $\mathbf{x}$ with a sentence $\mathbf{y}$. It splits into two steps.

1. Recognize the visual patterns in the image with a CNN g_ϕ (Chapter 7).
2. Generate a sentence conditioned on the image with an RNN language model.

The model is

$$f_\theta(\mathbf{y} \mid g_\phi(\mathbf{x})) :$$

the image features $g_\phi(\mathbf{x})$ condition the language model that produces $\mathbf{y}$, e.g. “man in black shirt is playing guitar.” The lecture listed naive captioning and captioning with attention as the two variants.